ARTEA: Theory-Guided Hierarchical Graph Index for High-Performance ANN Search

Weitang Ye
weitang.ye@ntu.edu.sg
Nanyang Technological University
Singapore

Dingheng Mo
dingheng001@e.ntu.edu.sg
Nanyang Technological University
Singapore

Siqiang Luo
siqiang.luo@ntu.edu.sg
Nanyang Technological University
Singapore

ABSTRACT

Approximate Nearest Neighbor (ANN) search is a critical infrastructure for modern data management and AI applications. While hierarchical proximity graphs have achieved widespread empirical success due to their multi-layer, coarse-to-fine routing efficiency, they remain less understood from a theoretical perspective: can hierarchical structures be endowed with both strict accuracy guarantees and low ANN search complexity simultaneously? In this paper, we answer affirmatively by introducing ARTEA, a novel hierarchical proximity graph. By integrating deterministic r-net sampling with a novel Aspect-Ratio-Constrained Pruning (ARC-PRUNING) technique, ARTEA bounds the worst-case search complexity to $O((\alpha \cdot \tau)^{\lambda} + \alpha^{\lambda} \log \Delta)$ under the practical τ-bounded scenario, where τ is a practical constant, λ is the doubling dimension of the metric space, and Δ is the dataset aspect ratio. This makes ARTEA the first proximity graph to achieve a strictly logarithmic dependence on Δ. Besides theoretical contributions, we further develop a robust system implementation inspired by the theoretical principles of ARTEA. Extensive empirical evaluations demonstrate that our approach outperforms prevailing baselines in search efficiency and result accuracy, while maintaining highly competitive index construction costs.

PVLDB Reference Format:
Weitang Ye, Dingheng Mo, and Siqiang Luo. ARTEA: Theory-Guided Hierarchical Graph Index for High-Performance ANN Search. PVLDB, 20(1): XXX-XXX, 2027. doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/NTU-Siqiang-Group/Artea.

1 INTRODUCTION

Approximate Nearest Neighbor (ANN) search in high-dimensional spaces is a crucial infrastructure for modern data management. It has been applied in ubiquitous applications including information retrieval [20], recommendation systems [41], large language models [4, 8, 17, 28, 31], and AI agents [43, 44]. Given a set of n data points $P \subset \mathbb{R}^d$ and a query vector $q \in \mathbb{R}^d$, our goal is to construct an index structure that allows a search algorithm to efficiently retrieve the point $p^* \in P$ closest to q. Among a diverse array of techniques to address the ANN problem, recent studies indicate that proximity graph based methods have emerged as the dominant solution, consistently demonstrating superior performance over classic approaches such as hashing techniques [14, 21, 32, 35, 49, 50], inverted indices [16, 39, 48], and tree-based structures [5, 6, 26, 27]. A *proximity graph* (PG) is a directed graph in which each vertex represents a data point, and edges are established according to *predefined criteria*, with the purpose of approximately locating the nearest neighbor of a query point by navigating the PG.

Within the family of PG-based methods [2, 11–13, 19, 24, 29, 34, 36, 37, 42, 45, 52, 53], several pioneering works [13, 23, 29, 34, 45] have conducted in-depth investigations into theoretically sound proximity graphs, as summarized in Table 1. A predominant paradigm among these solutions is employing pruning techniques to construct sparse graphs that support accurate ANN search through greedy routing. Intuitively, edge-pruning techniques start from a complete graph and prune a directed edge (u, v) if there exists another neighbor w of u that is sufficiently close to v, rendering the direct connection (u, v) redundant. This procedure is deterministic, as candidates are consistently processed in ascending order of their distance to u. Depending on the specific criteria used to define whether w and v are sufficiently close, various pruning strategies have emerged [13, 23, 29, 45], offering distinct trade-offs between search accuracy and query complexity. Among these methods, α-CG [29] represents the state of the art, achieving an exact retrieval guarantee with *polylogarithmic time complexity* under the practical setting where $\delta(p^*, q)$ is bounded. However, an inherent limitation of this approach is that its pruning process requires evaluating candidate vertices across the entire dataset. This global scope drastically inflates the per-step routing cost to $O(\log \Delta)$, where the dataset aspect ratio Δ acts as a scale-dependent variable , which describes the ratio between the maximum and minimum pairwise distance. Most recently, Lu and Tao [34] proposed an approach that exploits the spatial properties of a specific data structure (i.e., r-nets [27]) to accelerate graph construction. Nevertheless, this design still incurs an $O(\log \Delta)$ cost per routing step, leaving the overall query complexity with a polylogarithmic dependence on Δ.

In practice, these theoretically rigorous prototypes often incur prohibitively high construction costs [13, 24, 29, 45] or remain purely conceptual without implementations [34]. Consequently, these works typically introduce practical variants—heuristic approximations designed to translate these theoretical models into engineering-viable solutions. For instance, MRNG is approximated by the Navigating Spreading-out Graph (NSG), and the rigorous α-CG is relaxed to form the α-Convergent Neighborhood Graph (α-CNG). We also summarize the correspondence between these theoretical prototypes and their practical variants in Table 1.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

Table 1: Comparison of PG-based methods (under practical $\delta(q, v^*) < \tau\rho$ setting). The notations follow from [29].

Theoretical Prototype	Accuracy Guarantee	Routing Property	Average Complexity	Worst-case Complexity	Approximate Variant	Practical Performance‡	
						Querying	Construction
MRNG [13]	☑	Monotonic Path	$O(n^{\frac{2}{d}} \ln n)$	$O(n)$	NSG	1.00×	1.00×
Vamana [23]	✓	α-Shortcut Reachable	–	$O\left(\alpha^\lambda \log \Delta \log \frac{\Delta}{(\alpha-1)\epsilon}\right) \overset{\dagger}{\approx} O(\log^2 \Delta)$	Vamana [24]	1.60×	4.35×
G_{net} [34]	✓	Log Drop	–	$O\left((1/\epsilon)^\lambda \cdot \log^2 \Delta\right) \overset{\dagger}{\approx} O(\log^2 \Delta)$	–	–	–
τ-MG [45]	✔	τ-Monotonic Path	$O(n^{\frac{1}{d}} (\ln n)^2)$	$O(n)$	τ-MNG	1.39×	1.05×
α-CG [29]	✔	α-Reducible	–	$O((\alpha \cdot \tau)^\lambda \log \Delta \log_\alpha \Delta) \overset{\dagger}{\approx} O(\log^2 \Delta)$	α-CNG	1.68×	0.55×
ARTEA (Ours)	✔	Hierarchical Drop	–	$O\left((\alpha \cdot \tau)^\lambda + \alpha^\lambda \log \Delta\right) \overset{\dagger}{\approx} O(\log \Delta)$	**ARTEALIB**	3.31×	4.62×

✔: exact; ✓: $\left(\frac{\alpha+1}{\alpha-1} + \epsilon\right)$-ANN; ☑: guaranteed only when $\delta(q, v^*) = 0$; ×: no guarantee; †: asymptotic equality up to the constants α, τ, λ;
‡: **Querying** and **construction** performance (higher is better) is averaged across all evaluated datasets and normalized to NSG (see Sec. 6 for details).

Motivation. Theoretically sound, state-of-the-art indices [23, 29, 34, 45] have demonstrated strong efficiency in minimizing the number of routing steps required to locate the nearest neighbors. Their rapid search convergence is achieved by enforcing a proportional reduction in distance toward the query target at every hop. However, to guarantee such a substantial distance reduction property, the construction algorithm must evaluate candidate neighbors spanning the entire dataset when applying the tailored pruning inequalities. Consequently, the maximum out-degree of a vertex becomes inevitably entangled with $\log \Delta$, where Δ represents the dataset's aspect ratio (the ratio of the dataset's diameter to its minimum pairwise distance). This leads to inefficiency in index construction and the total search time to be at least $O((\log \Delta)^2)$.

Therefore, our core intuition is to structurally decompose a single expensive $O(\log \Delta)$ routing step into a sequence of lightweight, $O(1)$-cost "mini-steps". Executing a constant number of these mini-steps guarantees the same strict exponential decay in distance to the query, thereby bounding the total search time to $O(\log \Delta)$.

Solution. To realize the aforementioned theoretical properties, we adopt a hierarchical graph architecture. Traditional multi-tiered structures (such as HNSW) typically rely on random sampling to assign points to different layers. Instead, we aim to systematically co-design both the vertex hierarchy and the edge topology across the hierarchy to guarantee optimal search behaviors. Specifically, we deterministically populate the layers by constructing r-nets in a bottom-up fashion, securing a mathematically rigorous vertex distribution at each level. On top of this geometrically robust vertex hierarchy, we build the intra-layer proximity graphs using our novel ARC-PRUNING (Aspect-Ratio-Constrained Pruning) technique. By restricting the aspect ratio of every vertex's candidate neighbor list, ARC-PRUNING effectively limits per-layer out-degree of every vertex to a constant, ensuring that the cost of intra-layer routing remains $O(1)$ while preserving the critical navigability required to achieve exponential distance decay.

By integrating the deterministic bottom-up r-net hierarchy with ARC-PRUNING, our proposed structure, ARTEA, successfully advances the state-of-the-art in proximity graphs by reducing the search time complexity's dependence on the dataset aspect ratio Δ from polylogarithmic to strictly logarithmic. As summarized in Table 1, we present a rigorous theoretical comparison between ARTEA and prevailing PG-based methods under the practical τ-bounded scenario (τ is a practical constant, following [29, 45]). Crucially, ARTEA is the first proximity graph to achieve *a strictly logarithmic dependence on* Δ in search complexity under τ-bounded scenario and doubling metrics, surpassing all state-of-the-art flat indices. To further refine this bound, decoupling the pruning strategies between the upper and bottom layers allows ARTEA to transform the τ^λ term (λ is the doubling dimension which is treated as a constant) from a coupled multiplier into an isolated additive factor, yielding a worst-case search complexity of $O\left((\alpha \cdot \tau)^\lambda + \alpha^\lambda \log \Delta\right)$.

Finally, in line with existing works such as NSG [13], Vamana [24], τ-MNG [45], and α-CNG [29], we develop ARTEALIB, which adopts the core design principles of ARTEA while using efficient approximate construction techniques for practical scalability. Our extensive empirical evaluations show that ARTEALIB reduces index building cost while delivering strong query efficiency, consistently outperforming state-of-the-art baselines across the evaluated workloads.

Contributions. The contributions of this paper are as follows:

- **ARTEA: A Principled Hierarchical Architecture.** We introduce ARTEA, which systematically co-designs vertex distribution and edge topology. By integrating a deterministic r-net sampling with our novel ARC-PRUNING, ARTEA strictly limits the per-layer out-degree to a constant. This ensures $O(1)$ intra-layer routing costs while preserving optimal global navigability.
- **Rigorous Theoretical Guarantees.** ARTEA stands as the first proximity graph to achieve a strict logarithmic search dependence on Δ, under the practical τ-bounded scenario. By isolating the τ^λ overhead into an additive factor, we bound the worst-case complexity to $O\left((\alpha \cdot \tau)^\lambda + \alpha^\lambda \log \Delta\right)$, asymptotically better than state-of-the-art proximity graphs.
- **Robust System Implementation.** Bridging theory and practice, we implement ARTEA from scratch in over 9,000 lines of `C++` template code. Our implementation centers on a decoupled construction framework—incremental insertion for the sparse upper layers and a refinement engine for the massive bottom layer—whose high-performance infrastructure supports fast index construction.

2 PRELIMINARIES

This section defines the ANN search problem and introduces the notations used throughout our analysis. For ease of reference, Table 2 summarizes the main symbols. Readers may refer to our technical report [60] for a more comprehensive notation table.

2.1 Problem Definition and Notation

A metric space $(\mathcal{M}, \delta)$ is a set of points $\mathcal{M}$ endowed with a distance function $\delta: \mathcal{M} \times \mathcal{M} \to \mathbb{R}_{\geq 0}$, which satisfies the triangle inequality

Table 2: Main symbols and constants.

Notation	Description
Δ	Dataset aspect ratio: $d_{\max}/\rho$.
λ	Doubling dimension of the metric space.
α	Scaling parameter in the pruning condition.
τ	Workload bound $\delta(q,P) \le \tau\rho$ for greedy search; determines its upper-layer scales and the bottom pruning offset.
β	Geometric expansion factor ($\beta > 1$) between consecutive layer scales, i.e., $R_{h+1}/R_h = \beta$ for $h \ge 1$.
ψ_h	Layer target factor: $(\tau\rho + \sum_{i=0}^{h} R_i)/R_h$; use τ_k in place of τ for beam search.
Δ_h	Local aspect ratio threshold: $(\gamma+1)\psi_h + \gamma R_{h+1}/R_h$; beam search uses $\Delta_0 = (2\gamma+1)(1+\tau_k)$.

$\delta(u,w) \le \delta(u,v) + \delta(v,w)$ for any $u,v,w \in \mathcal{M}$. The distance between a point p and a set S is calculated by $\delta(p,S) = \min_{v\in S}\delta(p,v)$.

Consider a database $P \subseteq \mathcal{M}$ consisting of n points in d-dimensional space. For a query point q in $\mathcal{M}$, a point $p^* \in P$ is a ***nearest neighbor*** (NN) of q if $\delta(p^*,q) = \delta(q,P)$; while a point $p \in P$ is a $(1+\epsilon)$-***approximate nearest neighbor*** ($(1+\epsilon)$-ANN) of q for some constant $\epsilon > 0$ if $\delta(p,q) \le (1+\epsilon)\cdot\delta(p^*,q)$. Our objective is to develop a data structure that efficiently solves the ANN search problem with the aforementioned accuracy guarantee.

This paper focuses on the threshold-constrained setting where $\delta(q,P) \le \tau\cdot\rho$ for a user-defined constant τ, where ρ denotes the minimum pairwise distance in dataset P. This is motivated by the observation that in real-world benchmark workloads, queries are typically distinct from data points in P yet remain spatially close to their nearest neighbors [29, 45].

We note that in real-world scenarios, empirical studies often employ ranking-based metrics, such as *recall*, to compare the returned set with the true top-k results. Accordingly, $(1+\epsilon)$-ANN framework can be naturally generalized to approximate k-nearest neighbor (k-ANN) search problem by maintaining a sorted candidate list, known as *Beam Search* algorithm [13, 29, 45]. Furthermore, to analyze the complexity of Proximity Graphs (PGs), we utilize the framework of doubling metrics to quantify the intrinsic dimensionality of the data [6, 27], based on the following key notations:

Definition 2.1 (Aspect Ratio). Let $d_{\max}$ and ρ denote the diameter and the minimum pairwise distance of P, respectively. The *aspect ratio* of dataset P is defined as $\Delta = d_{\max}/\rho$.

Definition 2.2 (Doubling Dimension [6]). For any $p \in \mathcal{M}$ and $r \ge 0$, let $B(p,r) = \{x \in \mathcal{M} \mid \delta(p,x) \le r\}$ be the closed radius-r ball. The *doubling dimension* of $(\mathcal{M},\delta)$ is the smallest constant λ such that any ball $B(p,r)$ with $r > 0$ can be covered by at most 2^λ balls of radius $r/2$. Intuitively, the doubling dimension measures how rapidly a metric space expands across scales. For example, in the two-dimensional Euclidean plane, a radius-r disk can be covered by at most seven radius-$r/2$ disks, giving $\lambda \le \log_2 7 < 3$. Thus, the doubling dimension captures how rapidly a metric space expands as the observation scale increases.

Following recent theoretical frameworks [23, 29, 34], we assume the doubling dimension λ of our input dataset P is a small constant, i.e., $\lambda = O(1)$. The following is a fundamental property [34]:

Lemma 2.3. *Let $X \subset \mathbb{R}^d$ be a set with aspect ratio Δ_X. The cardinality of X is bounded by its aspect ratio, i.e., $|X| \le (\Delta_X)^{O(\lambda)}$.*

Since the intrinsic dimension λ is bounded, this inequality implies that if any subset X has a constant aspect ratio, then it contains only a constant number of points. In contrast, the global aspect ratio Δ of the entire input dataset P is typically non-constant and treated as a growing term with dataset size in complexity analysis.

2.2 Proximity Graphs

Proximity Graphs based indices have emerged as the state-of-the-art solution for ANN search in high-dimensional vector spaces. Formally, a PG is a directed graph $\mathcal{G}(\mathcal{V},\mathcal{E})$ where the set of vertices $\mathcal{V}$ corresponds to the original points in the database P and the edges $\mathcal{E}$ are carefully selected to facilitate the search algorithm to navigate the graph towards the nearest neighbor of q.

In the theoretical modeling of proximity graphs, an extensively studied routing procedure is ***Greedy Search*** [13, 23, 29, 34, 45], which iteratively hops from the current vertex p° to its closest out-neighbor with respect to q, terminating at a local minimum. Modern hierarchical indices [33, 37, 53] extend this paradigm into ***Hierarchical Greedy Search***. The hierarchy is organized into layers $\mathcal{G}_h$ induced over subsets $\mathcal{V}_h \subseteq \mathcal{V}_{h-1}$, with $\mathcal{G}_0$ containing the entire dataset. Starting from a global entry $p^\top$, the algorithm performs greedy routing at each level, passing the local minimum p° down as the starting point for the next. This top-down traversal guarantees a monotonically decreasing distance $\delta(p^\circ,q)$. Upper levels exploit long-range edges for rapid convergence, while $\mathcal{G}_0$ performs fine-grained refinement.

Intuitively, the query efficiency is dominated by the number of distance computations of each step (governed by vertex out-degrees) and the number of hops required to converge [13, 29, 45].

3 ARTEA GRAPH

3.1 Hierarchical Graph Structure

As illustrated in Figure 1(d), the *Artea Graph* is organized as a hierarchy of proximity graphs $\mathcal{G}$, defined as:

$$\mathcal{G} = \langle \mathcal{G}_0, \mathcal{G}_1, \ldots, \mathcal{G}_{h_{\max}-1}, \mathcal{G}_{h_{\max}} \rangle$$

where each layer $\mathcal{G}_h = (\mathcal{V}_h, \mathcal{E}_h)$ represents a distinct graph with vertex set $\mathcal{V}_h$ and edge set $\mathcal{E}_h$. Consistent with state-of-the-art hierarchical indexing schemes [37, 53], vertices for level h are selected from the preceding layer $\mathcal{V}_{h-1}$. This raises the following design question: *how should each layer $\mathcal{V}_h \subset \mathcal{V}_{h-1}$ be selected so that the resulting hierarchy supports the most effective navigation possible?*

Ideally, to function as an efficient navigational backbone, the upper-level vertices should be uniformly distributed to cover the underlying space. Popular hierarchical graph indices, such as HNSW [37] and MIRAGE [53], typically rely on randomized vertex promotion, where vertices are sampled into upper layers with a fixed probability. However, such oblivious probabilistic selection lacks deterministic spatial guarantees, exposing the hierarchy to two structural inefficiencies:

Navigational blind spots: A local region may contain no vertex promoted to the upper layer, forcing coarse-to-fine routing to take circuitous paths toward targets and reducing query efficiency.

Redundant clustering: Randomized promotion may select multiple nearby vertices, introducing spatial redundancy that weakens upper-layer sparsity and hinders efficient long-range traversal.

To explicitly circumvent these structural pitfalls, ARTEA Graph employs a deterministic selection strategy based on r-net [27], a fundamental tool in metric spaces, defined as follows:

Definition 3.1 (r-Net Selection Strategy). Given a subset $X \subseteq \mathcal{M}$ and a value $r > 0$, the *r-Net Selection Strategy* selects a subset $X_r \subseteq X$ satisfying the following two conditions: (1) Separation property. The pairwise distance between any two distinct points in X_r is at least r; (2) Covering property. For any point $x \in X$, there exists at least one point $x' \in X_r$ such that $\delta(x, x') \leq r$.

Let $\beta > 1$ be a constant, and define $R_0 = \rho$ and $R_h = (1+\tau)\rho\beta^{h-1}$ for $h \geq 1$. A sufficient height is $h_{\max} = \lceil \log_\beta \Delta \rceil + 1$. ARTEA constructs its hierarchy recursively as follows: the base layer is $\mathcal{V}_0 = P$, and for each level $h \geq 1$, the vertex set $\mathcal{V}_h$ is chosen as an R_h-net of $\mathcal{V}_{h-1}$. Since $R_{h_{\max}}$ exceeds the diameter of P, the top layer contains only one point. A direct consequence of this recursive construction is the *hierarchical covering property*, which guarantees that every point retains a nearby representative at every level of the hierarchy.

LEMMA 3.2 (HIERARCHICAL COVERING). *For any point $p \in P$ and any level $h \in \{0, 1, \ldots, h_{\max}\}$, there exists a vertex $u_h \in \mathcal{V}_h$ such that*

$$\delta(u_h, p) \leq \sum_{k=0}^{h} R_k = \rho + (1+\tau)\rho \sum_{k=1}^{h} \beta^{k-1}. \tag{1}$$

The bound follows by recursively applying the covering condition of r-nets along the hierarchy [27]. As we show in Section 4.2, this property is the key structural ingredient that enables logarithmic-time navigation, which justifies our choice of the r-Net Selection Strategy over the randomized sampling used by HNSW and MIRAGE. Meanwhile, because exact r-net construction can be expensive, in Section 5 we present an efficient approximation algorithm.

Having established the vertex sets for each level, we next construct the edges. Consistent with recent proximity graphs that offer theoretical guarantees, ARTEA Graph employs a specialized pruning strategy to determine out-edges for each vertex.

3.2 Aspect Ratio-Constrained Pruning

Recall that in the construction of proximity graphs, a critical step is to determine the connectivity for each vertex. In this task, the key is to retain shortcut vertices that navigate the search significantly closer to the query node. Flat indices such as α-CG [29] achieve this with theoretical guarantees through conservative pruning rules that preserve the edges needed for strict distance reduction. As illustrated in Figure 1a, however, this also leads to the retention of extra edges, including long-range outliers such as s and t, as well as points such as v that the inherent form of α-CG's pruning condition fails to exclude. Revisiting the pruning condition under ARTEA's hierarchical setting, however, yields a striking observation: the per-vertex edge budget can be drastically reduced — driving the maximum out-degree at every layer down to a constant. Two complementary mechanisms, both motivated by the hierarchical structure, together achieve this.

Mechanism 1 (Aspect Ratio Constraint): By examining the hierarchical routing process, we observe that the graph's *Hierarchical Covering Property* (Lemma 3.2) naturally bounds the local search space. Suppose the routing at the upper layer (h+1) successfully returns a valid approximate nearest neighbor, which then serves as the entry point for the current layer h. The covering property guarantees that the distance between this entry point and the target point in layer h is strictly bounded by a constant multiple of the current layer scale R_h. Consequently, this indicates that there exists a constant multiplier Δ_h, such that the entire single-layer routing path at level h is strictly confined within a local radius of $\Delta_h R_h$. Therefore, candidates outside this bounded range are strictly unnecessary for routing. By safely discarding these outliers, we bound the local candidate pool independently of the dataset's global aspect ratio Δ.

Algorithm 1: ARC-Pruning

Input: point p, candidate set C_h, minimum pairwise distance R_h, aspect ratio threshold Δ_h, conflicting ratio function $\mathcal{F}_{\text{AG}}$:$\mathcal{M}\times\mathcal{M}\to\mathbb{R}$
Output: a retained neighbor set $\mathcal{N}_h(p)$ of p at level h

1 ARC-candidate set $\widetilde{C}_h \leftarrow \{v \in C_h \mid \delta(p, v) \leq R_h\Delta_h\}$
2 sort $\widetilde{C}_h$ in ascending order of the distance to p
3 retained neighbor set $\mathcal{N}_h(p) \leftarrow \emptyset$
4 **for** *each* $u \in \widetilde{C}_h$ *(sorted)* **do**
5 $flag \leftarrow true$
6 **for** *each* $v \in \mathcal{N}_h(p)$ **do**
7 **if** $\delta(u, v) < \mathcal{F}_{AG}(p, u, h)$ **then**
8 $flag \leftarrow false$
9 **break**
10 **if** $flag = true$ **then**
11 $\mathcal{N}_h(p) \leftarrow \mathcal{N}_h(p) \cup \{u\}$
12 **return** $\mathcal{N}_h(p)$

Rather than evaluating the entire vertex set $\mathcal{V}_h$ as candidates—which inherently carries the large global aspect ratio Δ of the dataset—ARC-Pruning directly starts with a restricted candidate set. Specifically, at Line 1 of Algorithm 1, it filters the vertices right from the start by enforcing the local aspect ratio threshold Δ_h.

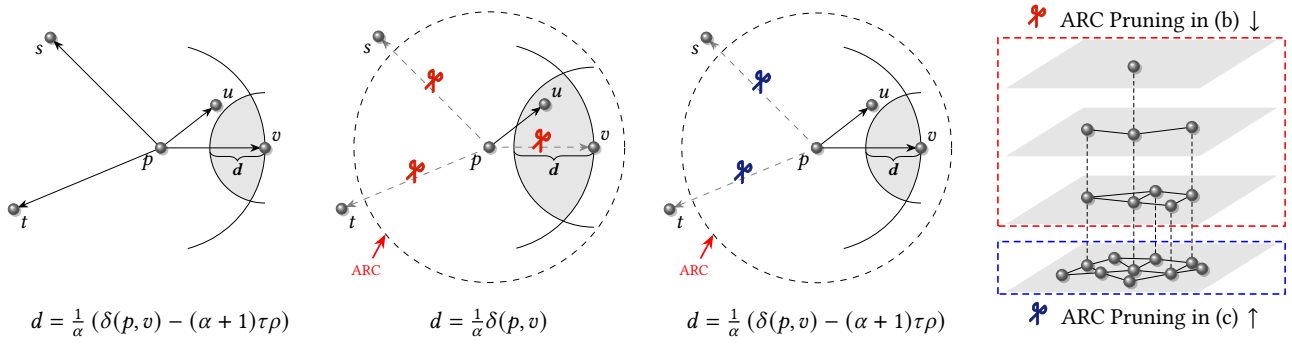
We introduce a dimensionless coefficient ψ_h such that $\psi_h R_h$ bounds the distance from the query to its nearest neighbor at level h. The pruning parameter $\alpha > 1$ controls pruning aggressiveness and determines the coefficient $(\alpha+1)/(\alpha-1)$ in the distance-reduction recurrence. The slack $\theta > 0$ adds a margin relative to $\psi_h R_h$ so that upper-layer search can stop after $O(1)$ hops for fixed parameters. Let $\gamma = \frac{\alpha+1}{\alpha-1} + \theta$ and define

$$\psi_h := \frac{\tau\rho + \sum_{i=0}^{h} R_i}{R_h}, \qquad \Delta_h := (\gamma+1)\psi_h + \gamma\frac{R_{h+1}}{R_h}. \tag{2}$$

Intuitively, $\Delta_h R_h$ defines how far routing needs to look from the current vertex. This range contains the nearest neighbor at level h, allowing vertices beyond it to be excluded from the current routing step. Section 4.1 formalizes this reasoning.

Using this threshold, ARC-Pruning effectively confines the candidates to a local region. As shown by the dashed boundary in Figures 1b and 1c, any candidates falling outside this restricted range, such as s and t, are universally pruned. Since $\psi_1 = 2$ and $\beta\psi_{h+1} = \psi_h + \beta$ for $h \geq 1$, the upper-layer factors satisfy

$$\Delta_h \leq (\gamma+1)\max\left\{2, \frac{\beta}{\beta-1}\right\} + \gamma\beta.$$



(a) A typical case in the pruning of SOTA PG (i.e., α-CG [29]). **(b) Aggressive ARC-Pruning for upper levels in ARTEA.** **(c) Conservative ARC-Pruning for the base level in ARTEA.** **(d) Overview of layerwise pruning scheme in ARTEA**

Figure 1: An Illustration of Graph Structure and Pruning Strategies. (a) SOTA flat indices (i.e., α-CG) use a conservative pruning condition to ensure strict distance reduction and exact query guarantee, retaining both vertex v and more distant vertices s, t. (b) In ARTEA Graph, at upper levels ($h \geq 1$), ARC-Pruning enforces a larger conflicting radius (gray area) to aggressively prune v for graph sparsity, while the aspect ratio constraint (dashed boundary) prunes s and t. (c) At the bottom level ($h = 0$), the conflicting radius is reduced to preserve critical edges like v for exact guarantee, while still filtering out s and t.

Thus the upper-layer normalized range is independent of τ, while $\Delta_0 = (2\gamma + 1)(1 + \tau)$. All normalized range factors are independent of the dataset size n and global aspect ratio Δ.

Mechanism 2 (Layer-wise Decoupled Pruning): Furthermore, since only the bottommost layer requires a high-precision search to locate the exact nearest neighbor, we can explicitly differentiate the pruning strategies between the upper and lower layers, as illustrated in Figure 1(d). For upper layers ($h \geq 1$), the pruning can be more aggressive, allowing the search to rapidly navigate into the local vicinity of a query point. Conversely, for the bottom layer ($h = 0$), the pruning must be more conservative, preserving enough edges to guarantee exact retrieval.

Lines 2 to 11 in Algorithm 1 specify this procedure. The algorithm first sorts the ARC-candidate set $\widetilde{C}_h$ in ascending order of distance to p. It then scans each candidate u and checks whether any previously retained neighbor lies within the conflicting radius of u (Line 7). This conflicting radius is defined by Eq. (3), with scaling parameter $\alpha = (c+\theta)/\theta$, where c is a constant:

$$\mathcal{F}_{\text{AG}}(p,u,h) = \begin{cases} \delta(p,u)/\alpha & \text{if } h \geq 1; \\ \frac{1}{\alpha}(\delta(p,u) - (\alpha+1)\tau\rho) & \text{if } h = 0. \end{cases} \tag{3}$$

This level-dependent radius elegantly trade-offs sparsity and precision. For upper levels ($h \geq 1$), ARC-Pruning uses the larger threshold $\delta(p,u)/\alpha$. This prunes candidates more aggressively and produces a sparser graph, as shown in Figure 1b. Such sparsity is sufficient because upper layers only need to find high-quality entry points. At the bottom level ($h = 0$), the bias term $(\alpha+1)\tau\rho$ reduces the conflicting radius, as shown in Figure 1c. This preserves more short-range edges, such as v, which are crucial for fine-grained exploration and exact nearest-neighbor convergence.

By decoupling the pruning logic across layers, we restrict the impact of the τ multiplier—which inflates the degree complexity in flat indices like α-CG—strictly to the bottommost layer. As will be demonstrated in the subsequent complexity analysis, this strategic confinement ultimately transforms τ from a global multiplier into an additive term in the query time. The following proposition clearly illustrates how this decoupling manifests in the out-degree bounds between the upper and bottom layers. We provide the detailed derivation in our technical report [60].

PROPOSITION 3.3. *The out-degree of any vertex p in layer h, denoted by $|\mathcal{N}_h(p)|$, is bounded as follows:*

$$|\mathcal{N}_h(p)| \leq \begin{cases} O\left(\alpha^\lambda \log \Delta_h\right) & \text{if } h \geq 1 \\ O\left((\alpha\tau)^\lambda + \alpha^\lambda \log \Delta_0\right) & \text{if } h = 0 \end{cases}$$

3.3 Space and Construction Time

Index Size Analysis. Based on Proposition 3.3, we analyze the total storage complexity. The index size is the cumulative cardinality of neighbor sets across all layers. With hierarchy height $h_{\max} = O(\log \Delta)$ and $|V| = n$, the total size is:

$$\mathcal{S}_{\text{AG}} = n \cdot \left(O\left((\alpha\tau)^\lambda + \alpha^\lambda \log \Delta_0\right) + \sum\nolimits_{h=1}^{h_{\max}} O\left(\alpha^\lambda \log \Delta_h\right)\right)$$

Given that $\log \Delta_h$ is bounded by a constant and $h_{\max}$=$O(\log \Delta)$, the final space complexity is $O\left(n \cdot \alpha^\lambda \cdot (\tau^\lambda + \log \Delta)\right)$. This confirms the index size scales linearly with n and logarithmically with Δ.

Construction Time. The construction procedure consists of two phases: *hierarchical vertex selection* and *hierarchical neighbor generation*. By employing the hierarchical net construction technique from [18], the vertex selection phase requires $O(n \log n)$ time. Next, we analyze the neighbor set generation (Algorithm 1) for each vertex p. Constructing the ARC-candidate set (Line 1) takes $O(n \log \Delta)$ time across all layers. The sorting procedure in Line 2 requires $O\left(\left|\widetilde{C}_h\right| \cdot \log |\widetilde{C}_h|\right)$. By Lemma 2.3, $|\widetilde{C}_h|$ is bounded by $O(\Delta_h^\lambda)$, which is independent of τ for $h \geq 1$ and is $O((1+\tau)^\lambda)$ for $h = 0$, with fixed $\alpha, \beta, \theta, \lambda$. Finally, the pruning procedure takes $O(|\widetilde{C}_h| \cdot |\mathcal{N}_h|)$. Thus, the processing time for each vertex is:

$$\begin{aligned} T_c(p) &= O(n \log \Delta) + \sum\nolimits_{h=0}^{h_{\max}} \left(O(|\widetilde{C}_h| \log |\widetilde{C}_h|) + O(|\widetilde{C}_h| \cdot |\mathcal{N}_h|)\right) \\ &= O(n \log \Delta) + \sum\nolimits_{h=0}^{h_{\max}} O(1) = O(n \log \Delta). \end{aligned}$$

The second equality holds because the terms $|\widetilde{C}_h|$ and $|\mathcal{N}_h|$ are bounded by structural constants (α, β, τ) independent of dataset scales n and Δ. Summing over all n vertices, the total construction time complexity is dominated by the linear scan for each vertex,

resulting in $T_c(\text{total}) = \sum_p T_c(p) = O(n^2 \log \Delta)$. This is comparable to that of existing methods such as Vamana, τ-MG, and α-CG, which generally require $\Omega(n^2)$ construction time. Since an $\Omega(n^2)$ construction cost can be prohibitive for large-scale datasets, we propose a more efficient, practical implementation in Section 5 that substantially accelerates the index building process while preserving the query performance of the Artea Graph.

4 THEORETICAL ANALYSIS OF QUERYING

The goal of this section is to show that the proposed construction supports both accurate routing and efficient search. In particular, we will show that the hierarchy and ARC-Pruning rule together guarantee exact nearest-neighbor retrieval under the τ-bounded setting, with only logarithmic dependence on the dataset aspect ratio Δ. Our analysis focuses on the Hierarchical Greedy Search framework introduced in Section 2, but with a *refined early-stopping rule*. The search proceeds from the top layer to the bottom layer. Upper layers only need to find high-quality entry points for the next layer, so they do not require full local convergence. Instead, once the routing at level h reaches a vertex v satisfying $\delta(v, q) \le \gamma\psi_h R_h$, it descends to $\mathcal{G}_{h-1}$.

4.1 Accuracy Guarantee

In this subsection, we show why Artea Graph's routing process is theoretically effective and reliable. The key observation is that whenever the current vertex is not good enough for its layer, then ARC-Pruning ensures that there is still an outgoing neighbor closer to the query. Hence, we can prove that Artea's search would never stop at a false local minima, and it eventually reaches high-quality entry points in upper layers and the exact nearest neighbor at the bottom layer. To facilitate the proof, we first establish several lemmas that characterize the properties of our hierarchical structure under practical settings. Let q be a query point whose exact nearest neighbor $p^* \in P$ satisfies $\delta(q, p^*) \le \tau\rho$. Furthermore, let p_h^* denote the exact nearest neighbor of q within the vertex set $\mathcal{V}_h$, and let $\hat{p}_h \in \mathcal{V}_h$ denote a vertex retrieved at level h that satisfies $\delta(\hat{p}_h, q) \le \gamma\psi_h R_h$. This vertex $\hat{p}_h$ serves as a high-quality entry point for routing in the subsequent layer.

Lemma 4.1. *For any $0 \le h \le h_{\max}$, given that $\delta(p^*, q) \le \tau\rho$, the distance between the exact nearest neighbor in layer $\mathcal{V}_h$ and the query point q satisfies:*

$$\delta(p_h^*, q) \le \psi_h \cdot R_h, \tag{4}$$

where ψ_h is defined in Eq. (2).

Proof. For every $h \ge 0$, Lemma 3.2 gives a representative $u_h \in \mathcal{V}_h$ of p_0^*, hence $\delta(p_h^*, q) \le \delta(u_h, q) \le \tau\rho + \sum_{i=0}^{h} R_i = \psi_h R_h$. □

Lemma 4.2. *For any $0 \le h < h_{\max}$, the relationship between the approximate nearest neighbor $\hat{p}_{h+1}$ at level h+1 and the exact nearest neighbor p_h^* at level h satisfies:*

$$\delta(\hat{p}_{h+1}, q) + \delta(p_h^*, q) \;\le\; \Delta_h \cdot R_h, \tag{5}$$

where Δ_h is defined in Eq. (2).

Proof. By definition, $\psi_{h+1}R_{h+1} = \psi_h R_h + R_{h+1}$. The entry guarantee and Lemma 4.1 therefore give, for every $h \ge 0$, $\delta(\hat{p}_{h+1}, q) + \delta(p_h^*, q) \le \gamma\psi_{h+1}R_{h+1} + \psi_h R_h = (\gamma+1)\psi_h R_h + \gamma R_{h+1} = \Delta_h R_h$. □

Building on Lemma 4.1 and Lemma 4.2, we present the main theorem regarding search accuracy guarantee.

Theorem 4.3 (Accuracy Guarantee). *For any query q whose exact nearest neighbor p_0^* satisfies $\delta(p_0^*, q) \le \tau\rho$, greedy routing on $\mathcal{G}_h$ guarantees the following: for levels $h \ge 1$, it terminates at a high-quality vertex $\hat{p}_h$ satisfying $\delta(\hat{p}_h, q) \le \gamma\psi_h R_h$; at the bottom level $h = 0$, it successfully converges to the exact NN p_0^*.*

Proof. We prove this theorem by induction on the level h, proceeding in a top-down manner, and the special scenario at the bottom level will be discussed in the induction step.

For the base case ($h = h_{\max}$), since the top level of the Artea Graph contains a single point $p^\top$, the routing process trivially returns $\hat{p}_{h_{\max}} = p^\top$, which coincides with $p^*_{h_{\max}}$ and serves as the entry point for the next level. By Lemma 4.1, $\delta(p^\top, q) \le \psi_{h_{\max}} R_{h_{\max}} \le \gamma\psi_{h_{\max}} R_{h_{\max}}$, where the second inequality holds since $\gamma > 1$ for any $\alpha > 1$ and $\theta \ge 0$. This establishes the base case.

For the induction step, we assume that greedy routing on level h+1 ($h \ge 0$) correctly returns a high-quality vertex $\hat{p}_{h+1}$ satisfying $\delta(\hat{p}_{h+1}, q) \le \gamma\psi_{h+1}R_{h+1}$. Consequently, the routing on level h initializes at $\hat{p}_{h+1}$. Let p be the current vertex visited in $\mathcal{G}_h$. If the exact nearest neighbor (at this level) p_h^* is an out-neighbor of p, the algorithm naturally moves to p_h^*, ensuring correctness. Otherwise, p_h^* must have been removed from p's out-neighbor set $\mathcal{N}_h(p)$. We analyze this absence in two cases:

Case 1 (ARC Violation): p_h^* was excluded during the ARC-candidate construction (Algorithm 1, Line 1). By Lemma 4.2, we have:

$$\delta(p, p_h^*) \le \delta(p, q) + \delta(p_h^*, q) \le \delta(\hat{p}_{h+1}, q) + \delta(p_h^*, q) \le \Delta_h \cdot R_h,$$

where $\delta(p, q) \le \delta(\hat{p}_{h+1}, q)$ holds because routing on $\mathcal{G}_h$ starts from $\hat{p}_{h+1}$ (by the inductive hypothesis) and greedy routing monotonically reduce the distance to q. According to the construction rule, p_h^* satisfies the distance threshold and must have been included in the ARC-candidate set of p. Thus, we verify p_h^* is within the ARC-candidate range—such that its absence from the final neighbor set implies it must have been removed during the pruning phase, leading us to Case 2.

Case 2 (Pruning Rule Violation): p_h^* was pruned due to the pruning condition in Line 7. This implies the existence of out-neighbor p_{out} within the conflicting radius of p. We analyze this based on the level-specific pruning logic of Artea Graph.

(i) Upper Layers ($h \ge 1$). The pruning condition implies $\delta(p_{\text{out}}, p_h^*) \le \delta(p, p_h^*)/\alpha$. We proceed by contradiction. Suppose p is a local minimum but **not** satisfying $\delta(p, q) \le \gamma\psi_h R_h$. By the triangle inequality and the pruning condition:

$$\begin{aligned}\delta(p_{\text{out}}, q) \;&\le\; \delta(p_{\text{out}}, p_h^*) + \delta(p_h^*, q) \;\le\; \frac{\delta(p, p_h^*)}{\alpha} + \delta(p_h^*, q) \\ &\le\; \frac{\delta(p, q)}{\alpha} + (1 + \frac{1}{\alpha}) \cdot \delta(p_h^*, q).\end{aligned} \tag{6}$$

By Lemma 4.1, we have, $\delta(p_h^*, q) \le \psi_h \cdot R_h \le \frac{\delta(p,q)}{\gamma}$. Substituting the bound for $\delta(p_h^*, q)$, we derive:

$$\delta(p_{\text{out}}, q) < \delta(p, q)\left[\frac{1}{\alpha} + \frac{\alpha+1}{\alpha\gamma}\right].$$

Provided that $\alpha > 1$ and $\theta > 0$, the term in the brackets is strictly less than 1. This yields $\delta(p_{\text{out}}, q) < \delta(p, q)$, contradicting the assumption that p is a local minimum. Thus, routing continues until a good enough $\hat{p}_h$ is found.

(ii) Bottom Layer ($h = 0$). At the bottom level, the pruning condition is relaxed using an offset $(\alpha + 1)\tau\rho$. A point p_0^* is pruned by p_{out} only if there is a neighbor p_{out} that occluded p_0^*, which implies $\delta(p_{\text{out}}, p_h^*) \le \frac{1}{\alpha}\left(\delta(p, p_h^*) - (\alpha+1)\tau\rho\right)$.

We proceed by contradiction. Assume the current vertex p is a local minimum but is **not** the exact nearest neighbor (i.e., $p \neq p_0^*$). Since p_0^* is pruned in this case, the pruning condition at the bottom layer implies the existence of an out-neighbor p_{out} of p, such that $\delta(p_{\text{out}}, p_0^*) \le \frac{1}{\alpha}\left(\delta(p, p_0^*) - (\alpha+1)\tau\rho\right)$. By combining this with the triangle inequality $\delta(p_{\text{out}}, q) \le \delta(p_{\text{out}}, p_0^*) + \delta(p_0^*, q)$, we derive:

$$\begin{aligned}
\delta(p_{\text{out}}, q) &\le \delta(p_{\text{out}}, p_0^*) + \delta(p_0^*, q) \\
&\le \frac{1}{\alpha}\left(\delta(p, p_0^*) - (\alpha+1)\tau\rho\right) + \delta(p_0^*, q) \\
&\le \frac{1}{\alpha}\left(\delta(p, q) + \delta(q, p^*) - (\alpha+1)\tau\rho + \alpha\delta(p_0^*, q)\right) \\
&\le \frac{1}{\alpha}\left(\delta(p, q) - \alpha\tau\rho + \alpha\delta(p_0^*, q)\right) \le \delta(p, q)/\alpha \qquad (7)
\end{aligned}$$

Since $\alpha > 1$, this inequality implies $\delta(p_{\text{out}}, q) < \delta(p, q)$. This directly contradicts the assumption that p is a local minimum, as there exists a neighbor p_{out} strictly closer to q. Consequently, the routing cannot stop at any $p \neq p_0^*$ and must terminate at the exact NN.

Conclusion: The greedy routing algorithm is guaranteed to converge effectively. For upper layers ($h \ge 1$), the strict angular pruning enforces a monotonic descent until a $(1+\theta)$-approximation is reached; this approximation suffices as a high-quality entry point for the subsequent layer. In contrast, for the bottom layer ($h = 0$), the relaxed pruning logic effectively eliminates all false local minima, thereby guaranteeing that the routing terminates at the exact nearest neighbor p_0^*. □

4.2 Query Time Complexity

In this subsection, we demonstrate that the Artea Graph achieves superior query time complexity compared to all other proximity graphs with theoretical guarantees. Intuitively, the time complexity of ANN search on a PG is governed by two primary factors: the routing length (i.e., the number of hops during traversal) and the out-degree of each vertex. Consequently, our theoretical analysis proceeds by proving that the routing path length is logarithmically bounded, and then deriving the total time complexity.

Proposition 4.4 (Routing Steps Bound). *The greedy routing on $\mathcal{G}_h$ terminates in $O(1)$ steps.*

Proof. Let $\pi = (v_0, v_1, \ldots, v_\ell)$ denote the sequence of vertices visited by the greedy routing at level h. The routing initializes at $v_0 = \hat{p}_{h+1}$, which is the entry point passed down from the upper level. As established in Theorem 4.3, the termination criterion differs by layer: for upper levels ($h \ge 1$), early termination is triggered once $\delta(v_\ell, q) \le \gamma\psi_h R_h$; for the bottom level ($h = 0$), the routing converges to the exact nearest neighbor $v_\ell = p_0^*$. In both cases, our objective is to bound the path length ℓ.

We first establish a distance-reduction invariant. From the initial condition specific to our hierarchical routing and Lemma 4.1, the distance bound for v_0 expands as:

$$\delta(v_0, q) \le \delta(\hat{p}_{h+1}, q) \le \gamma \cdot \psi_{h+1} \cdot R_{h+1}. \qquad (8)$$

Applying the inductive proof techniques from prior work [23, 29] to inequalities (6) and (7) under this initial condition (8), we deduce the following lemma:

Lemma 4.5. *For any level $0 \le h < h_{\max}$ and any step $i < \ell$ during routing, the distance from the current vertex v_i to the query q satisfies:*

$$\delta(v_i, q) \le \begin{cases} \dfrac{\gamma\,\psi_{h+1}\,R_{h+1}}{\alpha^i} + (\gamma - \theta)\,\delta(p_h^*, q), & h \ge 1 \qquad (9) \\ \dfrac{\gamma\,\psi_1\,R_1}{\alpha^i}, & h = 0. \qquad (10) \end{cases}$$

With this recurrence in hand, we bound ℓ at each level.

Upper Levels ($h \ge 1$). The routing on $\mathcal{G}_h$ descends to $\mathcal{G}_{h-1}$ once $\delta(v_i, q) \le \gamma\psi_h R_h$. By Lemma 4.1 and Eq. (9),

$$\delta(v_i, q) \le \frac{\gamma\psi_{h+1}R_{h+1}}{\alpha^i} + (\gamma - \theta)\psi_h R_h.$$

It suffices to reduce the first term to the slack $\theta\psi_h R_h$:

$$\frac{\gamma\psi_{h+1}R_{h+1}}{\alpha^i} \le \theta\psi_h R_h \iff \alpha^i \ge \frac{\gamma\beta\psi_{h+1}}{\theta\psi_h} \overset{(\dagger)}{=} \frac{\gamma(\psi_h + \beta)}{\theta\psi_h}$$

where step $(\dagger)$ uses the algebraic identity $\beta\psi_{h+1} = \psi_h + \beta$. Since $\psi_h \ge 1$ by Eq. (2), the threshold is bounded by $\gamma(1+\beta)/\theta$, so $i \ge \log_\alpha(\gamma(1+\beta)/\theta) = O(1)$ steps suffice to trigger the early termination rule.

Bottom Level ($h = 0$). Since $\psi_1 = 2$, the bottom-layer entry bound is $\gamma\psi_1 R_1 = 2\gamma(1+\tau)\rho$. The greedy routing on the bottom level must terminate at p_0^*. For every step that does not directly reach $p_0^\star$, the bottom-layer pruning rule enforces the *α-reducible property* (Theorem 4.3, Case 2(ii)): $\delta(v_{i+1}, q) \le \delta(v_i, q)/\alpha$. Hence, for every $i < \ell$, iterating from $v_0 = \hat{p}_1$ with $\delta(v_0, q) \le \gamma\psi_1 R_1$ yields Eq. (10). We split on the magnitude of $\delta(p_0^*, q)$.

Case A: $\delta(p_0^*, q) \ge \rho/2$. Since p_0^* is an exact nearest neighbor of q, applying Eq. (10) at $i = \ell - 1 < \ell$ gives

$$\frac{\rho}{2} \le \delta(p_0^*, q) \le \delta(v_{\ell-1}, q) \le \frac{\gamma\psi_1 R_1}{\alpha^{\ell-1}} \implies \ell \le 1 + \log_\alpha \frac{2\gamma\psi_1 R_1}{\rho}.$$

Case B: $\delta(p_0^*, q) < \rho/2$. By the r-net separation of $\mathcal{V}_0 = P$, every $v \in P \setminus \{p_0^*\}$ satisfies $\delta(v, p_0^*) \ge \rho$, so $\delta(v, q) > \rho/2$. Hence $\delta(v_{\ell-1}, q) > \rho/2$, and Eq. (10) gives

$$\ell \le \log_\alpha \frac{2\delta(v_0, q)}{\rho} + 1 \le \log_\alpha \frac{2\gamma\psi_1 R_1}{\rho} + 1.$$

As $\alpha, \beta, \gamma, \psi_1, \tau$ are constants, $\ell = O(1)$ in both cases.

Therefore, in both scenarios, the total path length is $\ell = O(1)$. □

By synthesizing Propositions 3.3 and 4.4, we determine the total time complexity of the routing procedure across all layers, thereby establishing the main theoretical result.

Theorem 4.6. *Let $\mathcal{G}$ be an Artea Graph constructed on a dataset P of size n. For any query point q with exact NN $p^* \in P$, if $\delta(q, p^*) \le \tau\rho$ for a user-defined parameter $\tau > 0$, the hierarchical greedy routing on $\mathcal{G}$ guarantees retrieval of p^* in $O\left((\alpha\tau)^\lambda + \alpha^\lambda \log \Delta\right)$ time.*

Discussion. For k-NN search, let $\tau_k \ge \tau$ satisfy $\delta(q, p_k^*) \le \tau_k\rho$, and construct the hierarchy with $R_0 = \rho$ and $R_h = (1+\tau_k)\rho\beta^{h-1}$ for $h \ge 1$. We replace τ by τ_k in Eq. (2), while retaining τ in the pruning rule (3); queries use upper-layer greedy routing followed by bottom-layer beam search.

Theorem 4.7 (Beam Search on Artea). *On the exact Artea graph, beam search with $L \geq k$ returns the exact top-k neighbors if*

$$L \geq |P \cap B(q, r_k)|, \qquad r_k = \frac{\alpha+1}{\alpha-1}(\tau_k - \tau)\rho. \tag{11}$$

Moreover, using the sufficient beam width, the number of vector distance computations is

$$O\Big(\log \Delta + \tau_k^{\lambda(\lambda+1)} \log \tau_k\Big),$$

which reduces to $O(\log \Delta)$ for fixed workload and index parameters.

Proof Sketch. If a true top-k neighbor is missing, ARC-Pruning guarantees that either the neighbor itself or a closer pruning witness has already been visited. The beam-width condition forces such a vertex to rank within the final beam, yielding a contradiction. The complexity follows from doubling-metric packing; see our technical report [60]. □

5 ARTEALIB: ENHANCING PRACTICAL EFFICIENCY WITH ARTEA GRAPH

Although the exact Artea graph provides rigorous guarantees, directly constructing it is prohibitively expensive at scale. Therefore, ArteaLib does not aim to construct an exact Artea graph with its theoretical guarantees. Instead, it adopts Artea's core design principles and uses approximate construction to achieve scalability and performance in practical settings.

5.1 Approximate Construction

To accelerate Artea index construction while preserving search quality, we adopt a decoupled framework that separates the construction methodologies of the upper layers and the bottom layer. Since increment-based approaches require a massive candidate pool for neighbor selection, causing severe build-time bottlenecks [42, 53], we confine such incremental construction to the upper layers, whose cardinality is orders of magnitude smaller. There, we build an approximate dynamic r-net that efficiently handles insertions and exploits the evolving topology to determine layer-specific insertion criteria. In contrast, for the bottom layer encompassing all data points, we delegate the construction to a highly optimized, refinement-based paradigm that rapidly refines an initial graph into a search-efficient topology.

Increment-based Construction of Approximate Hierarchical r-Nets (Upper Layers). Algorithm 2 uses $R_1 = (1+\tau)\rho$, or $R_1 = (1+\tau_k)\rho$ for the top-k configuration in Section 4.2. In the standard construction of hierarchical r-nets, to strictly enforce the covering and separation properties, a point p is promoted to layer h ($h > 0$) if and only if p satisfies $p \in \mathcal{V}_{h-1} \wedge \min_{v \in \mathcal{V}_h} \delta(p, v) > R_h$. Since enforcing this exactness is computationally prohibitive, we propose a novel approach for building high-quality approximate hierarchical r-nets. Specifically, we relax the insertion condition, substituting the exact minimum with an approximate nearest neighbor obtained via graph routing:

$$p \in \mathcal{V}_{h-1} \wedge \delta(p, \mathsf{NNSearch}(C_{h+1}, p, \mathcal{G}_h)) > R_h.$$

Within this scheme, the upper layers are constructed by streaming pointwise insertion (Algorithm 2). To determine each point's insertion level h_p, we run a fast top-down BeamSearch with a moderate budget L_1, approximating the nearest-neighbor distance d_h^*

Algorithm 2: Approximate r-Net Dynamic Insertion

Input: Hierarchical graph $\mathcal{G}$, new point p, radii $R_0 = \rho$ and coefficient τ_k, expansion base β, beam search widths L_1, L_2; $R_h = R_1 \beta^{h-1}$ for $h \geq 1$

1 $h_{\max} \leftarrow$ current highest layer of $\mathcal{G}$
2 $C_{h_{\max}+1} \leftarrow$ at most L_1 random vertices at top layer
3 **for** $h = h_{\max}$ ***down to*** 1 **do**
4 $\quad C_h \leftarrow \mathsf{BeamSearch}(C_{h+1}, p, \mathcal{G}_h, L_1)$
5 $\quad d_h^* \leftarrow \min_{v \in C_h} \mathrm{dist}(p, v)$
6 **if** $d_h^* > R_h$ *for all* $1 \leq h \leq h_{\max}$ **then**
7 $\quad h' \leftarrow 1 + \left\lceil \log_\beta \left(d^*_{h_{\max}} / R_1 \right) \right\rceil$
8 $\quad$ Extend the top layer to h'; $h_{\max} \leftarrow h'$
9 $h_p \leftarrow \min \left\{ h \in [0, h_{\max} - 1] \mid d_{h+1}^* \leq R_{h+1} \right\}$
10 **for** $h = h_p$ ***down to*** 1 **do**
11 $\quad$ Add p to $\mathcal{V}_h$
12 $\quad C_h \leftarrow \mathsf{BeamSearch}(C_h, p, \mathcal{G}_h, L_2)$
13 $\quad \mathcal{N}_h(p) \leftarrow \mathsf{ARC\text{-}Prune}(p, C_h, R_h)$
14 $\quad$ **forall** $v \in \mathcal{N}_h(p)$ **do**
15 $\quad\quad \mathsf{AddReverseEdge}(v, p, h)$
`// Edge construction of bottom layer is deferred`
16 Add p to $\mathcal{V}_0$

at each layer (Lines 3–5). Upward extension copies the singleton top layer and its cached candidates and distances to each new layer (Lines 6–8). We set h_p one level below the lowest layer where a covering vertex is found (Line 9). For the active insertion layers from h_p down to 1, we avoid redundant distance computations by warm-starting the search: reusing the candidate sets C_h cached during routing as seeds, BeamSearch resumes with an expanded budget L_2 for higher neighborhood quality (Line 12). The expanded candidates are then distilled via ARC-Pruning (with layer-specific radius R_h) to yield the neighborhood $\mathcal{N}_h(p)$ (Line 13). Reverse edges are then added (Lines 15), while bottom-layer edges are deferred to refinement (Line 16).

Refinement-based Bottom Layer Construction. This phase establishes the bottom layer's massive edge set. Rather than the vertex-by-vertex insertion paradigm of the previous phase, we present an efficient graph construction framework that significantly reduces construction time. The core of our framework features an iterative refinement engine based on an efficient Bulk Synchronous Parallel (BSP) model. In each iteration's computation phase, the engine assigns every vertex p to a thread, which applies a predefined *updater* to process p and its out-neighbors. Since some updaters also modify other vertices' out-neighborhoods, we use a log table to buffer such cross-vertex updates. In the synchronization phase, all vertices pull their pending updates from the log table, after which the table is flushed. By avoiding the locking overhead of read-write conflicts, this design accelerates construction while offering a clean abstraction for customizing refinement. We instantiate three updaters for bottom layer refinement:

Triangle Updater. Inspired by RNN-Descent, the intuition is that when a candidate v for p's neighborhood is pruned for being close to an existing neighbor u, v is a strong candidate to become an

out-neighbor of u and can be recommended to u. The original RNN-Descent couples pruning (p, v) and recommending (u, v) under one coupled MRNG-based pruning condition. Tightening it to our novel pruning threshold $\mathcal{F}_{\text{AG}}$ makes pruning harder to trigger; such that fewer prunings yield fewer recommendations and a suboptimal graph. We therefore decouple the recommendation threshold from the pruning threshold, recovering the "missed" recommendations and yielding a well-connected graph.

Reverse Updater. For the processing vertex p, we log a reverse edge (u, p) for each out-neighbor u. These edges may meet the recommendation threshold, letting the Triangle Updater escape local optima and trigger further iterations; they also strengthen connectivity, improving recall for top-k queries.

ARC Updater. This updater enforces the Aspect Ratio Constraint of Section 1. As it mutates only processing vertex p's local state, it is thread-safe and free of contention overhead.

Orchestrating the three updaters yields the approximate ArteaLib bottom layer: each of the t_2 outer rounds runs t_1 Triangle Updater self-loops followed by a single Reverse Updater pass; once all t_2 rounds complete, the ARC Updater finalizes the result. We fix $t_1 = 4$ and $t_2 = 15$ throughout all experiments.

5.2 Discussion

5.2.1 Structural Analysis. Algorithm 2 preserves the covering property of r-nets through its insertion rule, as proved in our technical report [60]. Separation, however, depends on the accuracy of NNSearch at each layer. With a sufficiently large search budget L_1, the search rarely misses an existing covering point, which limits its redundant promotions. On `SIFT-1M`, for example, only about `0.4025%` of the points in each upper layer violate separation.

At the bottom layer, ArteaLib preserves Artea's conflict-based pruning on the candidates exposed to each ARC-Pruning invocation: any neighbor u that prunes an ARC-range candidate v of p satisfies the conflicting-radius condition and remains in the output. The Triangle Updater then recommends v as a candidate neighbor of u, so repeated refinement progressively exposes more candidates to subsequent pruning passes.

Section 6.5.1 compares exact Artea with ArteaLib on a small dataset in construction time, index size, routing hops, efficiency, and accuracy.

5.2.2 Complexity Analysis. We measure construction cost by the number of distance computations, excluding sorting, queue maintenance, and synchronization. Let E be the average number of vertices expanded per insertion per layer across both searches, D a uniform upper-layer degree bound, and D_0 a bound on bottom-layer neighborhood and candidate sizes throughout refinement. Per insertion and layer, BeamSearch costs $O(ED)$ and ARC-Pruning costs $O(L_2 D)$, where L_2 is the neighborhood-construction queue size, so n insertions over $O(\log \Delta)$ layers cost $O(nD(E + L_2) \log \Delta)$. At the bottom layer, the $t_1 t_2$ Triangle-Updater passes, each costing $O(nD_0^2)$, dominate the $O(n(t_2 + 1)D_0)$ cost of the Reverse and final ARC passes. Thus, $T_{\text{build}} = O\big(nD(E + L_2) \log \Delta + nt_1 t_2 D_0^2\big)$, compared with $O(n^2 \log \Delta)$ for exact construction. Since searches run only on the smaller upper-layer graphs, E is generally modest, and Figure 4 shows that bottom-layer refinement, the second term, typically dominates construction time.

5.2.3 Dynamic Updates. This work focuses on static datasets. A possible dynamic extension of ArteaLib could reuse the incremental upper-layer construction of Algorithm 2, which preserves hierarchical covering for fixed layer radii, and insert into the bottom layer by searching for candidate neighbors, applying ARC-Pruning, and adding reverse edges, as in HNSW [37]. In real-world data streams, however, changes in ρ and Δ may require more sophisticated mechanisms to maintain an approximation of the Artea structure, which we leave to future work.

6 EXPERIMENTS

6.1 Experimental Setup

Environment. Our experiments are conducted on a dual-socket server equipped with two Intel Xeon Gold 6326 CPUs @ 2.90GHz and 256GB DDR4 main memory. The system runs 64-bit Ubuntu 22.04.5 LTS, and all implementations are compiled using `g++14.3` with `-O3` optimization flag.

Datasets. We benchmark ArteaLib on eight real-world datasets (Table 3) spanning multiple modalities (image, text, and latent audio), with dimensionalities from 96 to 960 and scales from 1M to 10M. We also report local intrinsic dimensionality (LID) [30], where a higher LID indicates a more locally high-dimensional and generally harder nearest-neighbor search space. LID is averaged over 1,000 sampled queries using their exact 1,000-NN distances. For datasets without predefined queries, we randomly hold out 10,000 base vectors as the query set, and compute exact ground truths via FAISS [10] brute-force ℓ_2 search. Following prior ANN studies [1, 15, 22], we remove exact duplicate vectors during dataset preprocessing for all baselines.

Competing Methods. We compared Artea against eight state-of-the-art PG algorithms, grouped by overall topology. From the *hierarchical index* family, we included `HNSW` [37], `HCNNG` [52], `MIRAGE` [53], and `HVS` [33]. From the *flat index* family, we included `NSG` [13], `Vamana` [24], τ-`MG` [45], and α-`CG` [29]. All baseline implementations used publicly available source code.[1]

Parameter Setting. We followed the settings recommended in the original papers [29, 53] or previous benchmarks [55] whenever available. `HNSW` and `HVS` used $(M, efConstruction) = (32, 500)$; `HVS` additionally used $T = 6$, $\delta = 0.5$, and 10,000 training samples. `Vamana` used $(R, L, \alpha) = (70, 75, 1.2)$. `MIRAGE` used $(S, R, M) = (32, 4, 32)$ with 15 iterations, `NSG`, α-`CNG`, and τ-`MNG` shared $C = 500$ and initial graph degree 60 on SIFT-1M, 100 on GIST-1M, 300 on SpaceV-10M, and 200 otherwise. Their (R, L) values were $(50, 40)$ on SIFT-1M, $(70, 60)$ on GIST-1M and YahooMusic, and $(100, 100)$ otherwise. For α-CNG, $(\alpha_{\text{start}}, \Delta\alpha, \alpha_{\text{max}}) = (0.9, 0.05, 1.6)$, with degree threshold $T = 25$ on SIFT-1M, 35 on GIST-1M, and 50 otherwise;

Table 3: Summary of Datasets.

Dataset	Dim	#Base	#Query	LID	Δ	$H^\dagger$
SIFT-1M [25]	128	1,000,000	10,000	19.04	510.72	4
GIST-1M [25]	960	1,000,000	1,000	37.38	2,195.75	5
Crawl [38]	300	1,998,995	10,000	32.24	95.32	5
GloVe-100 [46]	100	1,286,614	10,000	39.45	439.68	5
YahooMusic [59]	300	1,822,179	1,000	15.91	614.21	5
Tiny-5M [51]	384	5,000,000	1,000	39.30	1.055×10^8	5
DEEP-10M [3]	96	10,000,000	29,316	21.84	6,007.66	4
SpaceV-10M [7, 47]	100	10,000,000	10,000	26.98	63.10	4

$\dagger$: H denotes the number of levels in the hierarchical graph constructed by ArteaLib.

[1] `HNSW` has multiple publicly available implementations; we used `hnswlib` (available at https://github.com/nmslib/hnswlib).

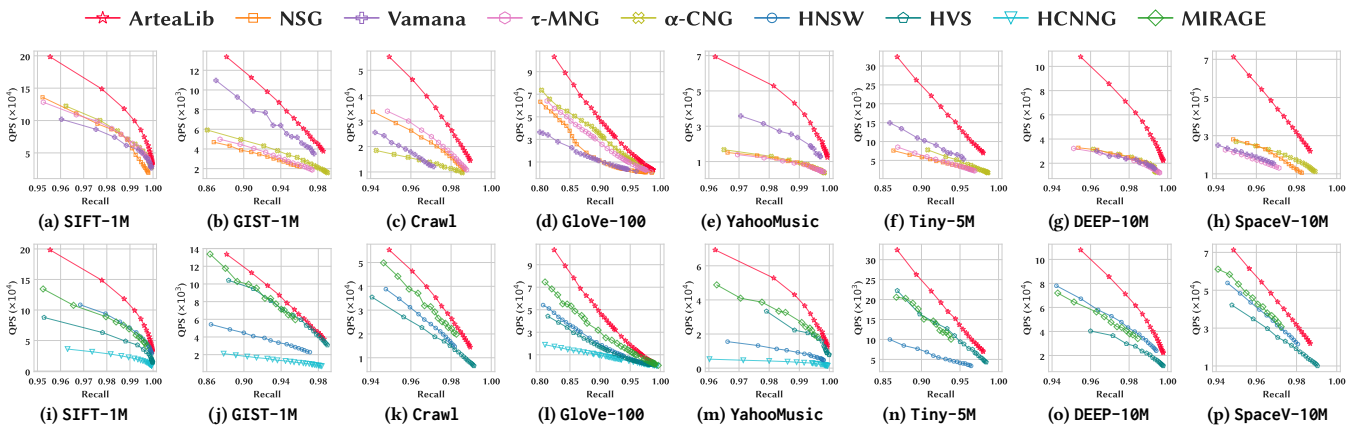


Figure 2: Recall@10 vs. QPS across eight datasets: flat graph baselines (top) and hierarchical graph baselines (bottom).

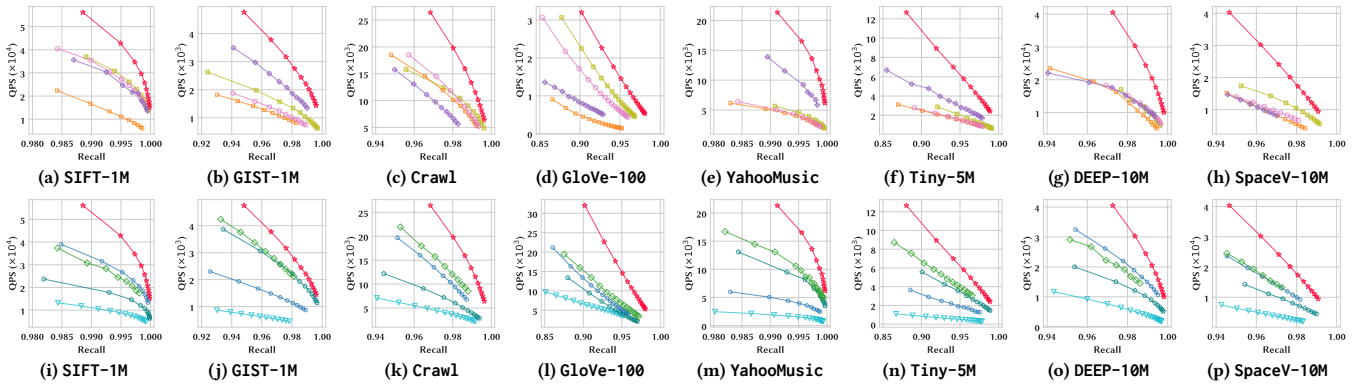


Figure 3: Recall@100 vs. QPS across eight datasets: flat graph baselines (top) and hierarchical graph baselines (bottom).

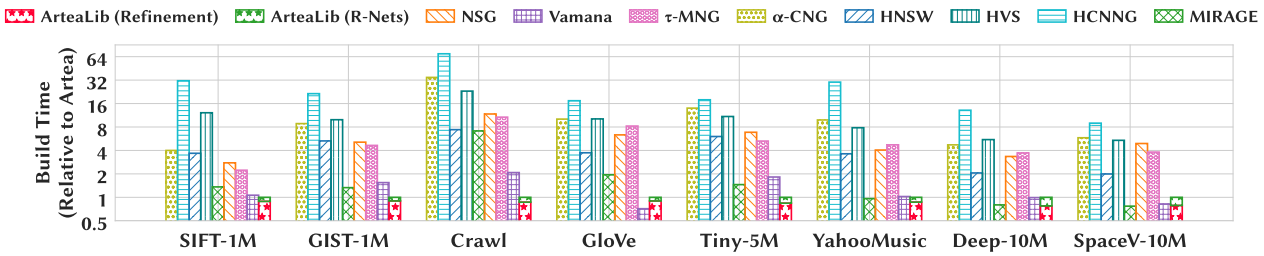


Figure 4: Index construction time comparison between ArteaLib and the baselines across all evaluated datasets.

τ-MNG used an angle threshold of 60°. Both used dataset-specific τ value. HCNNG used maximum MST degree 3 and cluster-size threshold $\lfloor \sqrt{N} + 0.5 \rfloor$. For Artea, we fix $\beta = 1.6, t_1 = 4, t_2 = 15, L_1 = 64, L_2 = 100$, and upper/base-layer degree limits to 32/64, with dataset-specific α, τ and τ_k. We swept the search beam width, or the distance-computation budget for HCNNG, to trace the recall–QPS Pareto frontiers.

6.2 Search Performance

We compare ArteaLib with the baselines using Recall@K versus QPS (Queries Per Second), with all methods using the maximum number of system threads. Recall@K is the fraction of true top-K neighbors retrieved, averaged over queries. We sweep the candidate queue budget to trace each method's recall–QPS trade-off; higher QPS at the same recall indicates faster search. Figures 2 and 3 show the recall–QPS trade-offs for Recall@10 and Recall@100, two settings commonly used in practice, across all eight datasets. In each figure, ArteaLib is compared with flat graph baselines in the upper row and hierarchical graph baselines in the lower row.

For $K = 100$, Table 4 summarizes the average and maximum QPS speedups of ArteaLib over the baselines with reported results, measured at dataset-specific target recall levels (i.e. 0.99 for `SIFT-1M`, `YahooMusic` and `Deep-10M`, 0.97 for `GIST-1M`, `Tiny-5M`, `Crawl`, and `SpaceV-10M`, 0.93 for `GloVe-100`).

These matched-recall comparisons quantify throughput differences without conflating search speed and retrieval accuracy.

6.3 Index Construction Performance

Construction Time. To clearly illustrate the relative efficiency, we normalize the construction time of Artea to 1 across all datasets,

Table 4: QPS speedup of ArteaLib over baseline methods.

Speedup	Hierarchical Family				Flat Family			
	HNSW	HCNNG	MIRAGE	HVS	NSG	Vamana	τ-MG	α-CG
Average	2.22×	6.19×	1.56×	2.28×	3.31×	2.07×	2.38×	1.97×
Maximum	3.50×	13.60×	2.20×	3.29×	8.46×	3.85×	4.70×	3.86×

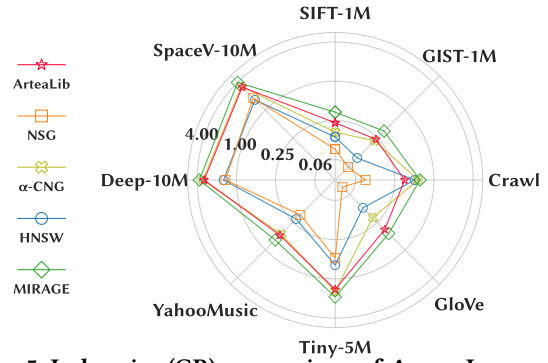


Figure 5: Index size (GB) comparison of ArteaLib against representative baselines.

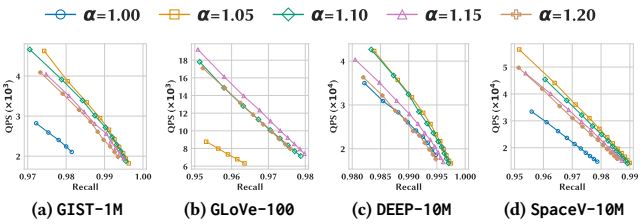


Figure 6: The impact of varying α on search performance.

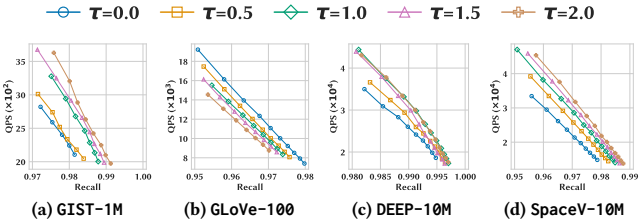


Figure 7: The impact of varying τ on search performance.

and report the execution time of other baselines as a relative factor (speedup or slowdown) compared to Artea. As depicted in Figure 4, the two hierarchical index structures (i.e., HNSW and HCNNG) typically demand substantially longer construction periods. Meanwhile, several proximity graphs backed by theoretically sound prototypes also suffer from heavy indexing overhead due to their complex pruning or selection phases. In contrast, Artea demonstrates remarkably fast construction speeds, achieving the optimal indexing efficiency in most cases. Additionally, as distinguished by the colors in Artea's bars, constructing the upper layers (i.e., approximate r-nets) consumes only a minor portion of the overall construction time.

Index Size. Regarding the memory overhead, we observe that Artea exhibits a larger index size compared to some flat baseline graphs. This is mainly attributed to its multi-layer hierarchical architecture; as illustrated in Figure 5, hierarchical graphs naturally require more edges to maintain connectivity across different layers.

Table 5: Performance comparison on a small dataset.

Metric	Exact Artea	ArteaLib
Construction time (s)	589.871	8.784
Index size (MiB)	108.14	73.64
p_{50} routing hops	7	13
p_{99} routing hops	9	21
Mean ADR	1.0000	1.0376
p_{50} NDC	423	617
p_{99} NDC	949	1 122

Even so, Artea remains more space-efficient than existing hierarchical baselines like MIRAGE. Nevertheless, given that proximity graphs typically occupy only a small fraction of the space required by raw vectors, we argue that this slight increase in index size is a highly worthwhile trade-off to unlock significantly superior QPS versus Recall Pareto performance.

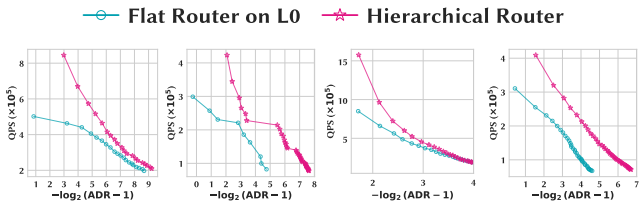
6.4 Effect of Parameters

This subsection explores the effects of α and τ in our pruning scheme (Section 3.2). We varied α from 1.00 to 1.20 and τ from 0.0 to 2.0 for each dataset. To evaluate the search performance across different parameter combinations, we tuned the runtime candidate queue size to reach a predefined target recall@10 and report the corresponding QPS. Consistent with observations in prior studies [29, 45], indefinitely increasing α and τ does not yield continuous gains. For a fixed degree bound, larger α and τ relax pruning, retaining more neighbors and short-range edges. Consequently, scaling these parameters introduces a nuanced trade-off between efficiency and accuracy: while the computational cost per routing step increases, the improved graph quality enables us to achieve higher recall under the same candidate queue budget.

6.5 Empirical Analysis of Graph Structure

6.5.1 Characterizing Exact Artea . We compare exact Artea with ArteaLib on a small enough dataset for exhaustive construction: MiniBooNE (129,597 distinct vectors, 50 dimensions). Exact construction builds the r-net hierarchy by farthest-point sampling, and scans each layer's complete point set for ARC candidates. ArteaLib uses approximate construction with an out-degree cap at both the upper and bottom layers. Within each dataset, both methods share the same structural parameters. Table 5 exposes a construction–routing trade-off. The results show that, owing to its approximate graph construction scheme, ArteaLib achieves lower index construction time and smaller index size than exact Artea. We evaluate search accuracy using ADR (Average Distance Ratio, where a lower value indicates results closer to the exact nearest neighbors). When both methods use greedy search, exact Artea graph yields higher performance than ArteaLib.

6.5.2 Ablation Study of Hierarchy. To assess the effect of Artea's hierarchy, we compare the full Artea Graph with a flat variant retaining only the bottom layer. We report the top-1 QPS–quality trade-off by sweeping the beam-search queue size L. Quality is measured by ADR, with the x-axis showing $-\log_{10}(\text{ADR} - 1)$ so that higher values indicate better accuracy. As shown in Figure 8, the hierarchical design provides a clear performance benefit. Even at matched maximum out-degree, Artea Graph achieves higher QPS than Artea-Flat throughout the accuracy range, confirming that Artea's hierarchy meaningfully accelerates routing.



(a) SIFT-1M (b) DEEP-10M (c) GloVe-100 (d) SpaceV-10M

Figure 8: ADR vs. QPS between ArteA and ArteA Layer 0.

6.5.3 Edge-Length Distributions. Figure 9 compares layer-wise edge lengths for GloVe-100. ArteaLib shows more distinct edge-length scales across layers, whereas HNSW exhibits greater overlap between adjacent layers. This is consistent with radius-based layer selection separating longer-range upper-layer connections from local bottom-layer links.

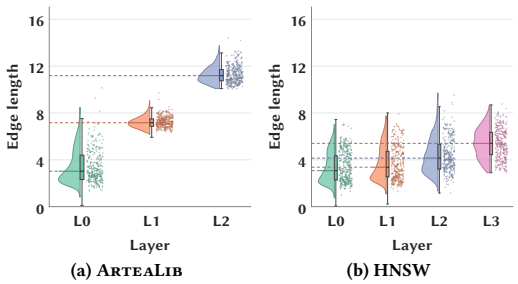
Discussion. Recent studies have questioned whether hierarchy is universally beneficial for graph-based ANNS [54, 56, 57]; in particular, removing HNSW's upper layers was found to cause little performance loss on several high-dimensional benchmarks, partly due to bottom-layer hub "highways" [40].

While long-range shortcuts are known to facilitate graph routing [58], our profiles suggest a complementary explanation: random level assignment does not enforce geometric separation, and overlapping edge-length scales may limit the additional navigation provided by HNSW's upper layers. ArteaLib's radius-based selection and ARC-Pruning promote a clearer division between long-range navigation and local exploration. Besides, we argue that the benefit of hierarchy is nevertheless workload-dependent: for larger k or higher-recall queries, extensive bottom-layer exploration can dominate, reducing the relative gain from upper layers.

7 RELATED WORK

While various techniques—such as hashing [14, 21, 32, 35, 49, 50], inverted indices [16, 39, 48], and tree structures [5, 6, 26, 27]—address the ANN problem, proximity graphs (PGs) [11–13, 19, 24, 29, 34, 36, 37, 42, 45, 52, 53] dominate due to consistently superior performance. Recent investigations into theoretically sound PGs [13, 23, 29, 34, 45] broadly classify them into two categories: *pruning-based* and *structural-driven* approaches.

Pruning-Based Approach. Methods in this category construct PGs through edge pruning strategies [13, 23, 29, 45], where formal variations in pruning criteria yield fundamentally different theoretical guarantees. The monotonic relative neighborhood graph (MRNG) [13] relies on the standard triangle inequality; however, it suffers from an $O(n)$ worst-case complexity and, more critically, fails to provide any search guarantees when $q \notin P$. To address this limitation, the τ-monotonic graph (τ-MG) [45] introduces a shifted term into the triangle inequality. Under the practical setting where the distance between a query q and its exact nearest neighbor v^* is strictly bounded by some constant, τ-MG guarantees exact retrieval; however, its worst-case time complexity is still bounded by $O(n)$. Subsequently, Indyk and Xu [23] demonstrated that the theoretical prototype of Vamana employs a scaled term in its pruning inequality. This adaptation allows Vamana to guarantee a $\left(\frac{\alpha+1}{\alpha-1}+\epsilon\right)$-approximate nearest neighbor (ANN) even for out-of-sample queries, establishing a poly-logarithmic worst-case search complexity of $O\left(\alpha^{\lambda}\log\Delta\log\frac{\Delta}{(\alpha-1)\epsilon}\right)$. Most recently, the α-convergent graph (α-CG) [29] synergized the scaled term from Vamana with the shifted term from τ-MG. Operating under the same practical setting, α-CG guarantees exact nearest neighbor retrieval within a worst-case complexity of $O\left((\alpha\cdot\tau)^{\lambda}\log\Delta\log_{\alpha}\Delta\right)$. While these methods design pruning inequalities to derive stronger accuracy guarantees and tighter query complexities, they are strictly confined to flat structures. Breaking this barrier, ArteA extends the rigorous pruning paradigm to complex hierarchical structures, achieving a significant improvement in query complexity.



(a) ArteaLib (b) HNSW

Figure 9: Edge-length distributions on GloVe-100

Structural-Driven Approach. The second category constructs proximity graphs by directly exploiting the spatial properties of r-nets [27], which does not rely on pruning-based paradigm, represented by G_{net} [34]. It directly leverages the r-net data structure, constructing multiple r-nets and establishing edges between each vertex v and points within specific ranges across these nets. By directly exploiting the structural properties of r-nets, G_{net} bypasses the edge pruning step, introducing a different methodology for fast proximity graph construction. Additionally, HENN [9] also adopts a hierarchical design, but uses range-space ϵ-nets, which are different from the r-nets. Its intra-layer navigable graph is treated as a black box, and its query-time analysis is correspondingly parameterized by the average degree d^* and a recall bound ρ_δ, yielding $O(d\cdot d^*\cdot\rho_\delta\cdot\log^2 n)$ query time. ArteA instead co-designs the hierarchy and intra-layer graph, providing an end-to-end exact-NN guarantee under the τ-bounded setting.

About r-Nets. G_{net} is the closest theoretical work to ours in its use of r-nets. While both ArteA and G_{net} draw inspiration from r-nets, their designs differ fundamentally in graph construction methods and routing strategies. ArteA enforces edge pruning within each layer and performs *hierarchical* greedy search, whereas G_{net} uses these nets mainly as a structural device for graph construction and still performs *flat* greedy routing. Consequently, G_{net}'s query complexity retains a polylogarithmic dependence on the global aspect ratio Δ. Note that earlier tree-based methods [6, 27] also utilized r-nets, but their routing mechanisms differ significantly from graph routing, placing them outside the scope of our discussion.

8 CONCLUSION

This paper presents ArteA, a principled hierarchical proximity graph with rigorous accuracy guarantees and low time complexities. ArteA is the first proximity graph to achieve a strictly logarithmic search complexity with respect to the dataset aspect ratio Δ, under practical assumptions. Furthermore, we translate the theoretical insights into a practical approach ArteaLib, achieving state-of-the-art empirical performance.

REFERENCES

[1] Akhil Arora, Sakshi Sinha, Piyush Kumar, and Arnab Bhattacharya. 2018. HD-index: pushing the scalability-accuracy boundary for approximate kNN search in high-dimensional spaces. *Proc. VLDB Endow.* 11, 8 (April 2018), 906–919. https://doi.org/10.14778/3204028.3204034

[2] Ilias Azizi, Karima Echihabi, and Themis Palpanas. 2023. ELPIS: Graph-Based Similarity Search for Scalable Data Science. *Proc. VLDB Endow.* 16, 6 (Feb. 2023), 1548–1559. https://doi.org/10.14778/3583140.3583166

[3] Artem Babenko and Victor Lempitsky. 2016. Efficient Indexing of Billion-Scale Datasets of Deep Descriptors. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. https://doi.org/10.1109/CVPR.2016.226

[4] Kai Uwe Barthel, Nico Hezel, Konstantin Schall, and Klaus Jung. 2019. Real-Time Visual Navigation in Huge Image Sets Using Similarity Graphs. In *Proceedings of the 27th ACM International Conference on Multimedia* (Nice, France) *(MM '19)*. Association for Computing Machinery, New York, NY, USA, 2202–2204. https://doi.org/10.1145/3343031.3350599

[5] Stefan Berchtold, Daniel A. Keim, and Hans-Peter Kriegel. 1996. The X-tree: An Index Structure for High-Dimensional Data. In *Proceedings of the 22th International Conference on Very Large Data Bases (VLDB '96)*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 28–39. https://dl.acm.org/doi/10.5555/645922.673502

[6] Alina Beygelzimer, Sham Kakade, and John Langford. 2006. Cover trees for nearest neighbor. In *Proceedings of the 23rd International Conference on Machine Learning* (Pittsburgh, Pennsylvania, USA) *(ICML '06)*. Association for Computing Machinery, New York, NY, USA, 97–104. https://doi.org/10.1145/1143844.1143857

[7] Qi Chen, Haidong Wang, Mingqin Li, Gang Ren, Scarlett Li, Jeffery Zhu, Jason Li, Chuanjie Liu, Lintao Zhang, and Jingdong Wang. 2018. *SPTAG: A library for fast approximate nearest neighbor search*. https://github.com/Microsoft/SPTAG

[8] Yaoqi Chen, Jinkai Zhang, Baotong Lu, Qianxi Zhang, Chengruidong Zhang, Jing Liu, Jingjia Luo, Di Liu, Huiqiang Jiang, Qi Chen, Bailu Ding, Xiao Yan, Jiawei Jiang, Chen Chen, Mingxing Zhang, Cheng Li, Yuqing Yang, Fan Yang, and Mao Yang. 2026. RetroInfer: A Vector Storage Engine for Scalable Long-Context LLM Inference. *Proc. VLDB Endow.* 19, 5 (May 2026), 1016–1031. https://doi.org/10.14778/3796195.3796212

[9] Mohsen Dehghankar and Abolfazl Asudeh. 2025. HENN: A Hierarchical Epsilon Net Navigation Graph for Approximate Nearest Neighbor Search. arXiv:2505.17368 [cs.DS] https://arxiv.org/abs/2505.17368

[10] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. 2026. The Faiss Library. *IEEE Transactions on Big Data* 12, 2 (2026), 346–361. https://doi.org/10.1109/TBDATA.2025.3618474

[11] Cong Fu and Deng Cai. 2016. EFANNA : An Extremely Fast Approximate Nearest Neighbor Search Algorithm Based on kNN Graph. arXiv:1609.07228 [cs.CV] https://arxiv.org/abs/1609.07228

[12] Cong Fu, Changxu Wang, and Deng Cai. 2022. High Dimensional Similarity Search With Satellite System Graph: Efficiency, Scalability, and Unindexed Query Compatibility. *IEEE Trans. Pattern Anal. Mach. Intell.* 44, 8 (Aug. 2022), 4139–4150. https://doi.org/10.1109/TPAMI.2021.3067706

[13] Cong Fu, Chao Xiang, Changxu Wang, and Deng Cai. 2019. Fast approximate nearest neighbor search with the navigating spreading-out graph. *Proc. VLDB Endow.* 12, 5 (Jan. 2019), 461–474. https://doi.org/10.14778/3303753.3303754

[14] Junhao Gan, Jianlin Feng, Qiong Fang, and Wilfred Ng. 2012. Locality-sensitive hashing scheme based on dynamic collision counting. In *Proceedings of the 2012 ACM SIGMOD International Conference on Management of Data* (Scottsdale, Arizona, USA) *(SIGMOD '12)*. Association for Computing Machinery, New York, NY, USA, 541–552. https://doi.org/10.1145/2213836.2213898

[15] Long Gong, Huayi Wang, Mitsunori Ogihara, and Jun Xu. 2020. iDEC: indexable distance estimating codes for approximate nearest neighbor search. *Proc. VLDB Endow.* 13, 9 (May 2020), 1483–1497. https://doi.org/10.14778/3397230.3397243

[16] Ruiqi Guo, Philip Sun, Erik Lindgren, Quan Geng, David Simcha, Felix Chern, and Sanjiv Kumar. 2020. Accelerating large-scale inference with anisotropic vector quantization. In *Proceedings of the 37th International Conference on Machine Learning (ICML'20)*. JMLR.org, Article 364, 10 pages. https://proceedings.mlr.press/v119/guo20h.html

[17] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang. 2020. REALM: retrieval-augmented language model pre-training. In *Proceedings of the 37th International Conference on Machine Learning (ICML'20)*. JMLR.org, Article 368, 10 pages. https://proceedings.mlr.press/v119/guu20a.html

[18] Sariel Har-Peled and Manor Mendel. 2006. Fast Construction of Nets in Low-Dimensional Metrics and Their Applications. *SIAM J. Comput.* 35, 5 (2006), 1148–1184. https://doi.org/10.1137/S0097539704446281

[19] Ben Harwood and Tom Drummond. 2016. FANNG: Fast Approximate Nearest Neighbour Graphs. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 5713–5722. https://doi.org/10.1109/CVPR.2016.616

[20] Jui-Ting Huang, Ashish Sharma, Shuying Sun, Li Xia, David Zhang, Philip Pronin, Janani Padmanabhan, Giuseppe Ottaviano, and Linjun Yang. 2020. Embedding-based Retrieval in Facebook Search. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining* (Virtual Event, CA, USA) *(KDD '20)*. Association for Computing Machinery, New York, NY, USA, 2553–2561. https://doi.org/10.1145/3394486.3403305

[21] Qiang Huang, Jianlin Feng, Yikai Zhang, Qiong Fang, and Wilfred Ng. 2015. Query-aware locality-sensitive hashing for approximate nearest neighbor search. *Proc. VLDB Endow.* 9, 1 (Sept. 2015), 1–12. https://doi.org/10.14778/2850469.2850470

[22] Qiang Huang and Anthony K. H. Tung. 2023. Lightweight-Yet-Efficient: Revitalizing Ball-Tree for Point-to-Hyperplane Nearest Neighbor Search. In *2023 IEEE 39th International Conference on Data Engineering (ICDE)*. 436–449. https://doi.org/10.1109/ICDE55515.2023.00040

[23] Piotr Indyk and Haike Xu. 2023. Worst-case performance of popular approximate nearest neighbor search implementations: guarantees and limitations. In *Proceedings of the 37th International Conference on Neural Information Processing Systems* (New Orleans, LA, USA) *(NIPS '23)*. Curran Associates Inc., Red Hook, NY, USA, Article 2891, 18 pages. https://doi.org/10.52202/075280-2891

[24] Suhas Jayaram Subramanya, Fnu Devvrit, Harsha Vardhan Simhadri, Ravishankar Krishnawamy, and Rohan Kadekodi. 2019. DiskANN: Fast Accurate Billion-point Nearest Neighbor Search on a Single Node. In *Advances in Neural Information Processing Systems*, H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett (Eds.), Vol. 32. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2019/file/09853c7fb1d3f8ee67a61b6bf4a7f8e6-Paper.pdf

[25] Herve Jegou, Matthijs Douze, and Cordelia Schmid. 2011. Product Quantization for Nearest Neighbor Search. *IEEE Trans. Pattern Anal. Mach. Intell.* 33, 1 (Jan. 2011), 117–128. https://doi.org/10.1109/TPAMI.2010.57

[26] Norio Katayama and Shin'ichi Satoh. 1997. The SR-tree: an index structure for high-dimensional nearest neighbor queries. *SIGMOD Rec.* 26, 2 (June 1997), 369–380. https://doi.org/10.1145/253262.253347

[27] Robert Krauthgamer and James R. Lee. 2004. Navigating nets: simple algorithms for proximity search. In *Proceedings of the Fifteenth Annual ACM-SIAM Symposium on Discrete Algorithms* (New Orleans, Louisiana) *(SODA '04)*. Society for Industrial and Applied Mathematics, USA, 798–807. https://research.ibm.com/publications/navigating-nets-simple-algorithms-for-proximity-search

[28] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Proceedings of the 34th International Conference on Neural Information Processing Systems* (Vancouver, BC, Canada) *(NIPS '20)*. Curran Associates Inc., Red Hook, NY, USA, Article 793, 16 pages. https://dl.acm.org/doi/abs/10.5555/3495724.3496517

[29] Binhong Li, Xiao Yan, and Shangqi Lu. 2026. Fast-Convergent Proximity Graphs for Approximate Nearest Neighbor Search. *Proc. ACM Manag. Data* 4, 1, Article 36 (April 2026), 24 pages. https://doi.org/10.1145/3786650

[30] Wen Li, Ying Zhang, Yifang Sun, Wei Wang, Mingjie Li, Wenjie Zhang, and Xuemin Lin. 2020. Approximate Nearest Neighbor Search on High Dimensional Data — Experiments, Analyses, and Improvement . *IEEE Transactions on Knowledge & Data Engineering* 32, 08 (Aug. 2020), 1475–1488. https://doi.org/10.1109/TKDE.2019.2909204

[31] Di Liu, Meng Chen, Baotong Lu, Huiqiang Jiang, Zhenhua Han, Qianxi Zhang, Qi Chen, Chengruidong Zhang, Bailu Ding, Kai Zhang, Chen Chen, Fan Yang, Yuqing Yang, and Lili Qiu. 2025. RetrievalAttention: Accelerating Long-Context LLM Inference via Vector Retrieval. In *Advances in Neural Information Processing Systems*, D. Belgrave, C. Zhang, H. Lin, R. Pascanu, P. Koniusz, M. Ghassemi, and N. Chen (Eds.), Vol. 38. Curran Associates, Inc., 54358–54385. https://doi.org/10.52202/085713-1816

[32] Yingfan Liu, Jiangtao Cui, Zi Huang, Hui Li, and Heng Tao Shen. 2014. SK-LSH: an efficient index structure for approximate nearest neighbor search. *Proc. VLDB Endow.* 7, 9 (May 2014), 745–756. https://doi.org/10.14778/2732939.2732947

[33] Kejing Lu, Mineichi Kudo, Chuan Xiao, and Yoshiharu Ishikawa. 2021. HVS: hierarchical graph structure based on voronoi diagrams for solving approximate nearest neighbor search. *Proc. VLDB Endow.* 15, 2 (Oct. 2021), 246–258. https://doi.org/10.14778/3489496.3489506

[34] Shangqi Lu and Yufei Tao. 2025. Proximity Graphs for Similarity Search: Fast Construction, Lower Bounds, and Euclidean Separation. *Proc. ACM Manag. Data* 3, 5, Article 280 (Nov. 2025), 25 pages. https://doi.org/10.1145/3767716

[35] Qin Lv, William Josephson, Zhe Wang, Moses Charikar, and Kai Li. 2007. Multi-probe LSH: efficient indexing for high-dimensional similarity search. In *Proceedings of the 33rd International Conference on Very Large Data Bases* (Vienna, Austria) *(VLDB '07)*. VLDB Endowment, 950–961. https://vldb.org/conf/2007/papers/research/p950-lv.pdf

[36] Yu Malkov, Alexander Ponomarenko, Andrey Logvinov, and Vladimir Krylov. 2013. Approximate nearest neighbor algorithm based on navigable small world graphs. *Information Systems* 45 (01 2013), 61–68. https://doi.org/10.1016/j.is.2013.10.006

[37] Yu A. Malkov and D. A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Trans. Pattern Anal. Mach. Intell.* 42, 4 (April 2020), 824–836. https://doi.org/10.1109/TPAMI.2018.2889473

[38] Tomas Mikolov, Edouard Grave, Piotr Bojanowski, Christian Puhrsch, and Armand Joulin. 2018. Advances in Pre-Training Distributed Word Representations. In *Proceedings of the Eleventh International Conference on Language Resources and Evaluation (LREC 2018)*, Nicoletta Calzolari, Khalid Choukri, Christopher Cieri, Thierry Declerck, Sara Goggi, Koiti Hasida, Hitoshi Isahara, Bente Maegaard, Joseph Mariani, Hélène Mazo, Asuncion Moreno, Jan Odijk, Stelios Piperidis, and Takenobu Tokunaga (Eds.). European Language Resources Association (ELRA), Miyazaki, Japan. https://aclanthology.org/L18-1008/

[39] Jason Mohoney, Devesh Sarda, Mengze Tang, Shihabur Rahman Chowdhury, Anil Pacaci, Ihab F. Ilyas, Theodoros Rekatsinas, and Shivaram Venkataraman. 2025. Quake: adaptive indexing for vector search. In *Proceedings of the 19th USENIX Conference on Operating Systems Design and Implementation* (Boston, MA, USA) *(OSDI '25)*. USENIX Association, USA, Article 9, 17 pages. https://www.usenix.org/conference/osdi25/presentation/mohoney

[40] Blaise Munyampirwa, Vihan Lakshman, and Benjamin Coleman. 2025. Down with the Hierarchy: The 'H' in HNSW Stands for "Hubs". In *The 1st Workshop on Vector Databases*. https://openreview.net/forum?id=OJwITuuU3h

[41] Shumpei Okura, Yukihiro Tagami, Shingo Ono, and Akira Tajima. 2017. Embedding-based News Recommendation for Millions of Users. In *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (Halifax, NS, Canada) *(KDD '17)*. Association for Computing Machinery, New York, NY, USA, 1933–1942. https://doi.org/10.1145/3097983.3098108

[42] Naoki Ono and Yusuke Matsui. 2023. Relative NN-Descent: A Fast Index Construction for Graph-Based Approximate Nearest Neighbor Search. In *Proceedings of the 31st ACM International Conference on Multimedia* (Ottawa ON, Canada) *(MM '23)*. Association for Computing Machinery, New York, NY, USA, 1659–1667. https://doi.org/10.1145/3581783.3612290

[43] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. 2024. MemGPT: Towards LLMs as Operating Systems. arXiv:2310.08560 [cs.AI] https://arxiv.org/abs/2310.08560

[44] Joon Sung Park, Joseph O'Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. 2023. Generative Agents: Interactive Simulacra of Human Behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology* (San Francisco, CA, USA) *(UIST '23)*. Association for Computing Machinery, New York, NY, USA, Article 2, 22 pages. https://doi.org/10.1145/3586183.3606763

[45] Yun Peng, Byron Choi, Tsz Nam Chan, Jianye Yang, and Jianliang Xu. 2023. Efficient Approximate Nearest Neighbor Search in Multi-dimensional Databases. *Proc. ACM Manag. Data* 1, 1, Article 54 (May 2023), 27 pages. https://doi.org/10.1145/3588908

[46] Jeffrey Pennington, Richard Socher, and Christopher Manning. 2014. GloVe: Global Vectors for Word Representation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Alessandro Moschitti, Bo Pang, and Walter Daelemans (Eds.). Association for Computational Linguistics, Doha, Qatar, 1532–1543. https://doi.org/10.3115/v1/D14-1162

[47] Harsha Vardhan Simhadri, George Williams, Martin Aumüller, Matthijs Douze, Artem Babenko, Dmitry Baranchuk, Qi Chen, Lucas Hosseini, Ravishankar Krishnaswamny, Gopal Srinivasa, Suhas Jayaram Subramanya, and Jingdong Wang. 2022. Results of the NeurIPS'21 Challenge on Billion-Scale Approximate Nearest Neighbor Search. In *Proceedings of the NeurIPS 2021 Competitions and Demonstrations Track (Proceedings of Machine Learning Research)*, Douwe Kiela, Marco Ciccone, and Barbara Caputo (Eds.), Vol. 176. PMLR, 177–189. https://proceedings.mlr.press/v176/simhadri22a.html

[48] Philip Sun, David Simcha, Dave Dopson, Ruiqi Guo, and Sanjiv Kumar. 2023. SOAR: improved indexing for approximate nearest neighbor search. In *Proceedings of the 37th International Conference on Neural Information Processing Systems* (New Orleans, LA, USA) *(NIPS '23)*. Curran Associates Inc., Red Hook, NY, USA, Article 141, 16 pages. https://doi.org/10.52202/075280-0141

[49] Yifang Sun, Wei Wang, Jianbin Qin, Ying Zhang, and Xuemin Lin. 2014. SRS: solving c-approximate nearest neighbor queries in high dimensional euclidean space with a tiny index. *Proc. VLDB Endow.* 8, 1 (Sept. 2014), 1–12. https://doi.org/10.14778/2735461.2735462

[50] Yufei Tao, Ke Yi, Cheng Sheng, and Panos Kalnis. 2009. Quality and efficiency in high dimensional nearest neighbor search. In *Proceedings of the 2009 ACM SIGMOD International Conference on Management of Data* (Providence, Rhode Island, USA) *(SIGMOD '09)*. Association for Computing Machinery, New York, NY, USA, 563–576. https://doi.org/10.1145/1559845.1559905

[51] Antonio Torralba, Rob Fergus, and William T. Freeman. 2008. 80 Million Tiny Images: A Large Data Set for Nonparametric Object and Scene Recognition. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 30, 11 (2008), 1958–1970. https://doi.org/10.1109/TPAMI.2008.128

[52] Javier Vargas Muñoz, Marcos A. Gonçalves, Zanoni Dias, and Ricardo da S. Torres. 2019. Hierarchical Clustering-Based Graphs for Large Scale Approximate Nearest Neighbor Search. *Pattern Recogn.* 96, C (Dec. 2019), 10. https://doi.org/10.1016/j.patcog.2019.106970

[53] Sairaj Voruganti and M. Tamer Özsu. 2025. MIRAGE-ANNS: Mixed Approach Graph-based Indexing for Approximate Nearest Neighbor Search. *Proc. ACM Manag. Data* 3, 3, Article 188 (June 2025), 27 pages. https://doi.org/10.1145/3725325

[54] Hongya Wang, Zhizheng Wang, Wei Wang, Yingyuan Xiao, Zeng Zhao, and Kaixiang Yang. 2020. A Note on Graph-Based Nearest Neighbor Search. *ArXiv* abs/2012.11083 (2020). https://api.semanticscholar.org/CorpusID:229340641

[55] Mengzhao Wang, Xiaoliang Xu, Qiang Yue, and Yuxiang Wang. 2021. A comprehensive survey and experimental comparison of graph-based approximate nearest neighbor search. *Proc. VLDB Endow.* 14, 11 (July 2021), 1964–1978. https://doi.org/10.14778/3476249.3476255

[56] Zeyu Wang, Peng Wang, Themis Palpanas, and Wei Wang. 2023. Graph- and Tree-based Indexes for High-dimensional Vector Similarity Search: Analyses, Comparisons, and Future Directions. *IEEE Data Eng. Bull.* 47 (2023), 3–21. https://api.semanticscholar.org/CorpusID:265509541

[57] Zeyu Wang, Qitong Wang, Xiaoxing Cheng, Peng Wang, Themis Palpanas, and Wei Wang. 2024. Steiner-Hardness: A Query Hardness Measure for Graph-Based ANN Indexes. *Proc. VLDB Endow.* 17, 13 (Sept. 2024), 4668–4682. https://doi.org/10.14778/3704965.3704974

[58] Duncan J. Watts and Steven H. Strogatz. 1998. Collective dynamics of 'small-world' networks. *Nature* 393 (1998), 440–442. https://api.semanticscholar.org/CorpusID:3034643

[59] Xiao Yan, Jinfeng Li, Xinyan Dai, Hongzhi Chen, and James Cheng. 2018. Norm-Ranging LSH for Maximum Inner Product Search. In *Advances in Neural Information Processing Systems*, S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett (Eds.), Vol. 31. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2018/file/b60c5ab647a27045b462934977ccad9a-Paper.pdf

[60] Weitang Ye, Dingheng Mo, and Siqiang Luo. 2026. Artea: Theory-Guided Hierarchical Graph Index for High-Performance ANN Search. https://github.com/NTU-Siqiang-Group/Artea/blob/main/artea-technical-report.pdf. Accessed: 2026-06-01.

A NOTATION TABLE

Table 6 summarizes the main symbols used in this paper.

Table 6: Main symbols and constants.

Notation	Description
P	Database consisting of n points in d-dimensional space.
n, d	Dataset size and ambient dimension, respectively.
$(\mathcal{M}, \delta)$	Metric space with point set $\mathcal{M}$ and distance function δ.
$\delta(p, S)$	Distance between a point p and a set S: $\min_{v \in S} \delta(p, v)$.
q; p^*, v^*	Query point; exact nearest neighbor of q in P.
k	Number of nearest neighbors requested.
$d_{\max}$, ρ	Diameter and minimum pairwise distance of P, respectively.
Δ	Dataset aspect ratio: $d_{\max}/\rho$.
λ	Doubling dimension of the metric space.
$B(p, r)$	Closed radius-r ball: $\{x \in \mathcal{M} \mid \delta(p, x) \le r\}$, for $r \ge 0$.
$\mathcal{G}$, $\mathcal{G}_h$	Hierarchy of proximity graphs and its graph at layer h, respectively.
$\mathcal{V}_h$, $\mathcal{E}_h$	Vertex set and edge set of $\mathcal{G}_h$, respectively.
$h_{\max}$	Hierarchy height, with $h_{\max} = \lceil \log_\beta \Delta \rceil + 1$.
β, R_h	Expansion base $\beta > 1$ and layer scales: $R_0 = \rho$, $R_h = (1+\tau)\rho\beta^{h-1}$ for $h \ge 1$; use τ_k in upper-layer scales for beam search.
$p^\top$, p°	Global entry point and current hop vertex, respectively.
p_h^*, $\hat{p}_h$	Exact nearest neighbor of q in $\mathcal{V}_h$ and approximate nearest neighbor returned at level h, respectively.
τ	Workload bound $\delta(q, P) \le \tau\rho$ for greedy search; determines its upper-layer scales and the bottom pruning offset.
τ_k	Top-k workload bound with $\tau_k \ge \tau$; determines the beam-search hierarchy scales and ARC ranges (Theorem 4.7).
α, c	Scaling parameter in the pruning condition, with $\alpha = (c+\theta)/\theta$ for a constant c.
θ	System constant related to hierarchical routing.
γ	Upper-layer approximation factor: $\frac{\alpha+1}{\alpha-1} + \theta$.
ψ_h	Layer target factor: $(\tau\rho + \sum_{i=0}^{h} R_i)/R_h$; use τ_k in place of τ for beam search.
Δ_h	Local aspect ratio threshold: $(\gamma+1)\psi_h + \gamma R_{h+1}/R_h$; beam search uses $\Delta_0 = (2\gamma+1)(1+\tau_k)$.
C_h, $\widetilde{C}_h$	Candidate set and ARC-candidate set, respectively; the latter retains candidates within distance $R_h\Delta_h$ of p. In beam search, C_h is the search candidate set at layer h.
L, L_h	Bottom-layer and layer-specific beam widths: $L_0 = L$ and $L_h = 1$ for $h \ge 1$.
H	Set of vertices expanded at the current search layer.
r_k	Beam comparison radius: $\frac{\alpha+1}{\alpha-1}(\tau_k - \tau)\rho$.
$\mathcal{N}_h(p)$	Retained out-neighbor set of p at level h.
$\mathcal{F}_{\mathrm{AG}}(p, u, h)$	Level-dependent conflicting radius defined in Eq. (3).

B AN EXAMPLE OF DOUBLING DIMENSION

Example. Consider the Euclidean plane $(\mathbb{R}^2, \|\cdot\|_2)$. Figure 10 illustrates how a ball $B(p, r)$ can be covered by seven balls of radius $r/2$: one centered at p, and six centered at the vertices of a regular hexagon at distance $\sqrt{3}r/2$ from p. The central ball covers the inner disk, while each outer ball covers the remaining part of a $60°$ sector. Translation and scaling give the same construction for any center and radius, establishing $\lambda \le \log_2 7$. More generally, doubling dimension is the base-2 logarithm of the smallest uniform bound on the number of half-radius balls needed to cover a ball.

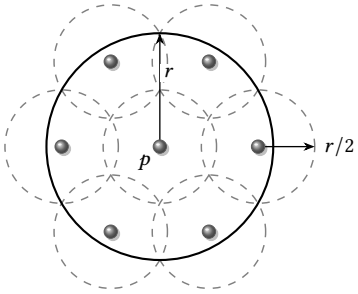


Figure 10: Covering a two-dimensional Euclidean ball of radius r with seven balls of radius $r/2$.

C PROOF OF PROPOSITION 3.3

Proposition 3.3. *The out-degree of any vertex p in layer h, denoted by $|\mathcal{N}_h(p)|$, is bounded as follows:*

$$|\mathcal{N}_h(p)| \le \begin{cases} O\left(\alpha^\lambda \log \Delta_h\right) & \text{if } h \ge 1 \\ O\left((\alpha\tau)^\lambda + \alpha^\lambda \log \Delta_0\right) & \text{if } h = 0 \end{cases}$$

Proof. In this section, we verify the space complexity (out-degree bound) stated in Proposition 3.3. Consider an arbitrary point p at layer h. We aim to bound the cardinality of its neighbor set $\mathcal{N}_h(p)$. The search range is bounded by $R_{\text{search}} = \Delta_h R_h$. We partition the search space around p into a sequence of disjoint concentric annuli $\mathcal{A}_i$, where the i-th annulus has an inner radius $\xi_i = 2^i R_h$ and covers the range $(\xi_i, 2\xi_i]$. The minimum pairwise separation σ_i within $\mathcal{A}_i$ is governed by the R_h-net property ($\sigma_i \ge R_h$) and the pruning condition defined in Eq. (3). We analyze the cardinality by distinguishing between upper layers and the bottom layer.

Scenario 1: Upper Layers ($h \ge 1$). For $h \ge 1$, the pruning condition requires $\delta(u, v) > \delta(p, u)/\alpha$. Thus, the separation in annulus $\mathcal{A}_i$ is bounded by:

$$\sigma_i \ge \max\left\{R_h, \frac{\xi_i}{\alpha}\right\}.$$

(i) Near Field ($\xi_i \le \alpha R_h$): Consider the range where $\xi_i \le \alpha R_h$. Here, the separation is dominated by R_h. The points of all these annulus reside in a volume of radius $O(\alpha R_h)$ with separation R_h. By the Lemma 2.3, the cardinality is:

$$|\mathcal{N}_h^{\text{near}}| \le O\left(\left(\frac{\alpha R_h}{R_h}\right)^\lambda\right) = O(\alpha^\lambda).$$

(ii) Far Field ($\xi_i > \alpha R_h$): Consider the range where $\xi_i > \alpha R_h$. Here, $\sigma_i \ge \xi_i/\alpha$. The aspect ratio of the annulus region is constant ($O(\alpha)$), meaning each annulus contributes at most $O(\alpha^\lambda)$ neighbors. Summing over the logarithmic range of annuli (bounded by $\log \Delta_h$):

$$|\mathcal{N}_h^{\text{far}}| \le \sum O(\alpha^\lambda) = O(\alpha^\lambda \log \Delta_h).$$

Result ($h \ge 1$): Summing both fields yields $|\mathcal{N}_h(p)| = O(\alpha^\lambda \log \Delta_h)$.

Scenario 2: Bottom Layer ($h = 0$). Recall $R_0 = \rho$ and set $C_0 = (\alpha+1)\tau\rho$. The pruning rule gives

$$\sigma_i \ge \max\left\{R_0, \frac{\xi_i - C_0}{\alpha}\right\}.$$

Define the threshold

$$T := \max\{\alpha R_0 + C_0,\ 2C_0\} = O(\alpha(1+\tau)R_0).$$

(i) Bounded Field ($\xi_i < T$): These annuli lie within distance $2T$ of p. Since $\mathcal{V}_0 = P$ is R_0-separated, Lemma 2.3 gives

$$|\mathcal{N}_0^{\text{bounded}}| = O\left((T/R_0)^\lambda\right) = O\left((\alpha(1+\tau))^\lambda\right).$$

(ii) True Far Field ($\xi_i \geq T$): Since $\xi_i \geq 2C_0$, we have

$$\sigma_i \geq \frac{\xi_i - C_0}{\alpha} \geq \frac{\xi_i}{2\alpha}.$$

Each annulus lies within distance $2\xi_i$ of p, so packing bounds its cardinality by $O(\alpha^\lambda)$. Summing over $O(\log \Delta_0)$ annuli gives $|\mathcal{N}_0^{\text{far}}| = O(\alpha^\lambda \log \Delta_0)$.

Result ($h = 0$): Summing both fields, using $(1+\tau)^\lambda = O(1+\tau^\lambda)$ for fixed λ and $\log \Delta_0 = \Theta(1)$ under fixed structural parameters to absorb the residual α^λ term, yields

$$|\mathcal{N}_0(p)| = O\left((\alpha\tau)^\lambda + \alpha^\lambda \log \Delta_0\right).$$

Conclusion. The out-degree is strictly bounded in both upper and bottom levels. For upper layers, it scales with $O(\alpha^\lambda \log \Delta_h)$, while for bottom layer, the relaxed pruning introduces an additive $(\alpha\tau)^\lambda$ term, resulting in $O((\alpha\tau)^\lambda + \alpha^\lambda \log \Delta_0)$. □

D PROOF OF THEOREM 4.7

We prove Theorem 4.7 for the exact hierarchy with complete ARC-candidate sets and no additional degree truncation, retaining the bottom-layer offset $(\alpha+1)\tau\rho$ in Eq. (3).

Parameters and search rule. Let $1 \leq k \leq n$, and choose $\tau_k \geq \tau$ such that every true top-k neighbor t satisfies $\delta(t,q) \leq \tau_k \rho$. Construct the hierarchy with $R_0 = \rho$ and $R_h = (1+\tau_k)\rho\beta^{h-1}$ for $h \geq 1$, with sufficient height $h_{\max} = \lceil \log_\beta \Delta \rceil + 1$. Use Eq. (2) with τ_k in place of τ:

$$\begin{aligned} \psi_h &= \frac{\tau_k \rho + \sum_{i=0}^{h} R_i}{R_h}, \\ \Delta_h &= (\gamma+1)\psi_h + \gamma \frac{R_{h+1}}{R_h}. \end{aligned} \tag{12}$$

Thus $\Delta_0 = (2\gamma+1)(1+\tau_k)$. The pruning function still uses τ. The covering argument requires only $\delta(p^*, q) \leq \tau_k \rho$: for every $h \geq 0$,

$$\delta(p_h^*, q) \leq \tau_k \rho + \sum_{i=0}^{h} R_i = \psi_h R_h.$$

Here $\psi_1 = 2$; for $h \geq 1$, $R_{h+1} = \beta R_h$ and $\beta\psi_{h+1} = \psi_h + \beta$. Thus the upper-layer arguments of Theorem 4.3 and Proposition 4.4 give $O(1)$ greedy hops per upper layer for fixed α, β, θ. The resulting bottom-layer entry satisfies

$$\delta(\hat{p}_1, q) \leq \gamma\psi_1 R_1 = 2\gamma(1+\tau_k)\rho. \tag{13}$$

Algorithm 3 gives the full search procedure. Its layer width is $L_h = 1$ for $h \geq 1$ and $L_0 = L$, while H records vertices expanded at the current layer. Upper layers stop once $\delta(q, C_h) \leq \gamma\psi_h R_h$; the bottom layer stops only when every current candidate has been expanded. Selecting the closest L_h points retains all points if fewer are available, with a fixed total order resolving distance ties consistently with the true top-k set. Within each layer, the worst beam rank never increases after the beam fills, so discarded points cannot reenter. Thus C_0 always contains the L closest visited vertices, or all visited vertices if fewer than L, as required below.

Algorithm 3: Hierarchical Beam Search

Input: graph $\mathcal{G}$, query q, entry $p^\top$, result size k, width $L \geq k$; $h_{\max}, R_h, \psi_h, \gamma$ as defined above
Output: the k closest candidates to q

$C_{h_{\max}} \leftarrow \{p^\top\}$
for *h from* $h_{\max}$ *down to* 0 **do**
  $L_h \leftarrow 1$ if $h \geq 1$, otherwise L
  $H \leftarrow \emptyset$
  while $C_h \setminus H \neq \emptyset$ **do**
    if $h \geq 1$ *and* $\delta(q, C_h) \leq \gamma\psi_h R_h$ **then break**
    $p^\circ \leftarrow \arg\min_{v \in C_h \setminus H} \delta(v, q)$
    $C_h \leftarrow$ closest L_h points to q in $C_h \cup \mathcal{N}_h(p^\circ)$
    $H \leftarrow H \cup \{p^\circ\}$
  if $h \geq 1$ **then**
    $\hat{p}_h \leftarrow$ the unique vertex in C_h
    $C_{h-1} \leftarrow \{\hat{p}_h\}$
return the k closest points to q in C_0

Theorem 4.7. *Under the above construction and search rule, exact top-k retrieval is guaranteed for any integer beam width $L \geq k$ satisfying*

$$L \geq |P \cap B(q, r_k)|,$$

where $r_k = \frac{\alpha+1}{\alpha-1}(\tau_k - \tau)\rho$.

Proof. Suppose the final candidate set C_0 misses a true top-k neighbor t, and choose $p \in C_0$ minimizing $\delta(p,t)$. By this choice and the closest-visited invariant, with $\hat{p}_1$ visited initially, Eqs. (12)–(13) give

$$\begin{aligned} \delta(p,t) &\leq \delta(q, C_0) + \delta(t,q) \\ &\leq \gamma\psi_1 R_1 + \tau_k \rho \\ &\leq \gamma\psi_1 R_1 + \psi_0 R_0 = \Delta_0 \rho. \end{aligned}$$

Hence t is an ARC candidate of p. Either $t \in \mathcal{N}_0(p)$, or Algorithm 1 retains a witness $p_{\text{out}} \in \mathcal{N}_0(p)$ with

$$\delta(p_{\text{out}}, t) < \frac{\delta(p,t) - (\alpha+1)\tau\rho}{\alpha} < \delta(p,t).$$

The last inequality implies $p_{\text{out}} \notin C_0$ by the choice of p. Moreover, if v is a farthest vertex of C_0 from q, the triangle inequality yields

$$\begin{aligned} \delta(p_{\text{out}}, q) &< \frac{\delta(p,q)}{\alpha} + \frac{\alpha+1}{\alpha}\big(\delta(t,q) - \tau\rho\big) \\ &\leq \frac{\delta(v,q)}{\alpha} + \frac{\alpha+1}{\alpha}\big(\delta(t,q) - \tau\rho\big). \end{aligned} \tag{14}$$

At termination, p has been expanded, so the direct target t or its witness has already been visited. If $|C_0| < L$, every visited vertex belongs to C_0, contradicting its absence. We may therefore assume $|C_0| = L$.

Since $L \geq k$ and t is missing, t precedes the worst-ranked member of C_0, and $\delta(t,q) \leq \delta(v,q)$. If $\delta(v,q) < r_k$, then $C_0 \cup \{t\} \subseteq P \cap B(q, r_k)$, contradicting $L \geq |P \cap B(q, r_k)|$. Hence $\delta(v,q) \geq r_k$.

If t is a direct neighbor of p, it is a visited vertex ranking before the worst member of C_0, a contradiction. Otherwise, using $\delta(t,q) \leq \tau_k\rho$ and the definition of r_k in Eq. (14) gives

$$\delta(p_{\text{out}}, q) < \frac{\delta(v,q)}{\alpha} + \frac{\alpha-1}{\alpha} r_k \leq \delta(v,q).$$

Thus the visited witness outside C_0 ranks before its worst member, again contradicting the definition of C_0. □

Beam width and query cost. For any $a \geq 0$, doubling covers $B(q, (1+a)\rho)$ by at most $[8(1+a)]^\lambda$ balls of radius at most $\rho/4$. Each contains at most one data point because P is ρ-separated. Consequently, a query-independent sufficient choice for Eq. (11) is

$$L = \min\left\{n, \max\left\{k, \left\lceil [8(1+r_k/\rho)]^\lambda \right\rceil\right\}\right\}. \tag{15}$$

We measure query cost by the number of vector distance computations. Fix $\alpha, \beta, \theta, \lambda$, with τ_k and Δ bounded below by constants greater than one. Then $\Delta_0 = (2\gamma+1)(1+\tau_k) = \Theta(\tau_k)$, so proposition 3.3 gives

$$|\mathcal{N}_0(p)| = O\big(\tau^\lambda + \log \tau_k\big).$$

The polynomial degree term remains governed by τ, while the dependence on τ_k is logarithmic.

Beam width alone does not bound cumulative expansions by L. Every bottom-layer edge has length at most $\Delta_0\rho$. Before the beam first fills, fewer than L vertices have been visited, so each is reachable from $\hat{p}_1$ in fewer than L edges. Vertices visited in the expansion that fills the beam are reachable in at most L edges. Thereafter, the farthest beam distance is nonincreasing. By Eq. (13), $\delta(\hat{p}_1, q) \leq \Delta_0\rho$, so every expanded vertex lies within distance $(L+1)\Delta_0\rho = O(L\tau_k\rho)$ of q. The same packing argument therefore bounds cumulative expansions by $O((L\tau_k)^\lambda)$. Each bottom-layer expansion evaluates at most $|\mathcal{N}_0(p)|$ vector distances. The upper-layer degrees and greedy hop counts are independent of τ_k, contributing $O(\log\Delta)$ distance computations. Multiplying the bottom-layer expansion bound by the degree bound and adding the upper-layer cost yields

$$T_{\text{query}} = O\Big(\log\Delta + (L\tau_k)^\lambda(\tau^\lambda + \log\tau_k)\Big). \tag{16}$$

This bound retains two effects of τ_k: it enlarges the region containing expanded vertices and increases the degree bound logarithmically. In addition, the sufficient width in Eq. (15) grows with $\tau_k - \tau$ through r_k.

For fixed k and τ, Eq. (15) gives $L = O(\tau_k^\lambda)$, since $1 + r_k/\rho = O(\tau_k)$. Substituting into Eq. (16) gives

$$T_{\text{query}} = O\Big(\log\Delta + \tau_k^{\lambda(\lambda+1)}\log\tau_k\Big). \tag{17}$$

This conservative packing bound makes the dependence on τ_k explicit; fixing τ_k further yields $O(\log\Delta)$ distance computations per query.

E PRESERVATION OF HIERARCHICAL COVERING

For fixed $\rho > 0$ and $\beta > 1$, let $R_0 = \rho$ and $R_h = R_1\beta^{h-1}$ for $h \geq 1$, where $R_1 = (1+\tau)\rho$, or $(1+\tau_k)\rho$ for top-k construction. We show that each completed insertion in Algorithm 2 preserves the covering property

$$\min_{u\in\mathcal{V}_h} \delta(x,u) \leq R_h, \qquad x \in \mathcal{V}_{h-1}, \quad 1 \leq h \leq h_{\max}. \tag{18}$$

Proof. We induct on insertions; the first point alone in each layer satisfies Eq. (18). Consider inserting p into a hierarchy satisfying Eq. (18), with top layer $\mathcal{V}_H = \{u_\top\}$. Existing points retain their covering representatives, so it suffices to check p. At the singleton top layer, BeamSearch evaluates $u_\top$, giving $d_H^* = \delta(p, u_\top)$. If no existing layer yields a covering point for p, the algorithm copies the top layer to each new layer up to

$$H' = 1 + \left\lceil \log_\beta \frac{\delta(p,u_\top)}{R_1} \right\rceil, \qquad \delta(p,u_\top) \leq R_{H'}.$$

The copies cover one another, and $u_\top$ covers p at layer H'.

After any extension, let j be the lowest layer where a point $v \in \mathcal{V}_j$ is found with $\delta(p,v) \leq R_j$. The test uses the exact distance $\delta(p,v)$, so approximate search may cause over-promotion by missing a covering point, but cannot falsely certify coverage. Line 9 sets $h_p = j-1$, so p is added to layers $0,\ldots,j-1$. Thus, p covers itself at every layer $1 \leq h < j$, and v covers it at layer j. This establishes the covering property for the new point and completes the proof. □